Innovative Infrastructure Solutions (2025) 10:198
https://doi.org/10.1007/s41062-025-01983-2

TECHNICAL PAPER

Concrete compressive strength classification using hybrid machine learning models and interactive GUI

Mostafa M. Alsaadawi[1,2] · Mohamed Kamel Elshaarawy[2] · Abdelrahman Kamal Hamed[2]

Received: 26 October 2024 / Accepted: 21 March 2025 / Published online: 28 April 2025
© The Author(s) 2025

Abstract
Concrete Compressive Strength (CCS) is a critical parameter in structural engineering, influencing durability, safety, and load-bearing capacity. This study explores the classification of CCS using hybrid Machine Learning (ML) techniques and an interactive Graphical User Interface (GUI). Advanced ML algorithms: Random Forest (RF), Adaptive-Boosting (AdaBoost), Extreme-Gradient-Boosting (XGBoost), Light-Gradient Boosting Machine (LightGBM), and Categorical-Boosting (CatBoost) were applied to categorize strength into Low, Normal, and High classes. The dataset, comprising 1298 samples, was split into 80% training and 20% testing for evaluation. Hyperparameter tuning was applied using Bayesian Optimization with fivefold stratified cross-validation, resulting greatly improved the model's performance. Results showed that LightGBM achieved the highest accuracy, with scores of 0.931 (Low), 0.865 (Normal), and 0.935 (High), and corresponding area under the curve values of 0.967, 0.938, and 0.981. CatBoost also performed well, particularly in Normal and High strength classes, while XGBoost showed slight overfitting in the Normal class. RF and AdaBoost had acceptable performance but struggled with boundary cases. To interpret model predictions, SHapley-Additive-exPlanations (SHAP) analysis was used. Curing duration and cement content were the most influential factors across all strength classes, while water content and superplasticizer played secondary roles. Coarse aggregate became more significant in High-Strength Concrete (HSC). A GUI was developed to allow practitioners to input data and receive real-time strength classifications, bridging the gap between machine learning and practical applications in concrete design.

Keywords Bayesian optimization · Classification · Concrete strength · Ensemble model · Graphical user interface · Cross-validation

Introduction

Rising greenhouse gas (GHG) emissions are having far-reaching consequences for Earth, one of the most noticeable of which is the melting of the Arctic and Antarctic polar ice caps [1]. This decline leads to rising sea levels and ecosystem disruptions, threatening biodiversity and human livelihoods. Building materials, especially concrete, are a leading source of GHG emissions; the construction industry is responsible for approximately half of all emissions worldwide [2]. Portland cement (PC), a key component of concrete, is responsible for approximately 7% of total global CO_2 emissions, primarily due to the calcination of limestone during its production [3, 4]. With global PC production expected to rise from 4000 million tons annually to 6000 million tons by 2060 [5], the environmental impact is set to worsen, underscoring the urgent need for sustainable alternatives [6–8]. A promising solution is the incorporation of industrial by-products such as ground granulated blast furnace slag (GGBS), granite powder, and fly ash into concrete [9–11]. These materials not only reduce GHG emissions potentially by up to 80% [12, 13], but also enhance concrete's durability and strength. Given that concrete is the most widely used construction material after water [14–17], optimizing its composition is crucial for

✉ Mostafa M. Alsaadawi
mostafa.m.alsaadawi@gmail.com

Mohamed Kamel Elshaarawy
melshaarawy@horus.edu.eg

Abdelrahman Kamal Hamed
akamal@horus.edu.eg

[1] Structural Engineering Department, Faculty of Engineering, Mansoura University, Mansoura 35516, Egypt

[2] Civil Engineering Department, Faculty of Engineering, Horus University-Egypt, New Damietta 34517, Egypt

sustainable development. The concrete compressive (CCS), a key performance indicator, depends on factors such as the water-to-binder ratio, aggregate properties, and binder type [18, 19]. The adoption of alternative binders might help to minimizing building's impact on the environment while maintaining or improving the structural integrity of concrete [20].

The conventional method for assessing CCS entails laboratory-based destructive testing. This method involves pulverizing concrete specimens of specified dimensions after a designated curing duration. This method is commonly considered a dependable approach for testing the load-bearing capacity of concrete materials [21]. Nonetheless, despite its precision, this testing method is increasingly regarded as inefficient due to its labor-intensive characteristics, high costs, and the necessity for specialized equipment. Furthermore, the destruction of samples during testing limits their reuse, thereby exacerbating material waste and overall costs. Due to the inefficiencies linked to destructive testing, researchers have investigated empirical regression models to estimate the CS of concrete based on the proportions of its mixture components [22–24]. These models utilize mathematical equations to forecast strength outcomes based on the ratios of components such as cement, water, aggregates, and admixtures. The precision of these models is limited by the complex, non-linear interactions among the mixture components. Crucial elements such as cement type and fineness, water-to-cement ratio, aggregate size and quality, and the inclusion of additives can substantially influence the ultimate strength, challenging accurate predictions [25].

Transformative changes have been brought to numerous fields, including the construction industry, by recent advancements in Artificial Intelligence (AI) and Machine Learning (ML). Various ML models, including Artificial Neural Networks (ANN), Support Vector Machines (SVM), Gene Expression Programming (GEP), and Deep Learning (DL), have exhibited remarkable proficiency in analyzing extensive datasets and revealing intricate patterns [26–32]. By utilizing extensive datasets of concrete mix compositions and the corresponding strength results, ML models can be trained in the field of concrete technology. This allows for more accurate strength predictions for novel concrete mixes to be achieved via predictive models than was previously possible with traditional regression methods. ML models excel at identifying intricate, non-linear relationships among tangible components, rendering them more efficient for strength prediction tasks, even when alternative materials such as fly ash are employed. Moreover, ML models can perpetually enhance their precision as additional data is gathered.

The environmental and performance benefits of fly ash, a by-product of coal combustion, have led to its growing recognition as a sustainable additive in concrete production. Concrete that contains it has better chemical resistance, lasts longer, and produces fewer waste materials for landfills. In order to enhance the use of ecologically friendly materials and optimize mixture designs that fit with performance standards, it is vital to use ML models for predicting the CS of fly ash-based concrete. This approach allows for both long-term structural stability and minimal impact on the environment. The water-to-binder ratio, aggregate size and quality, binder type, and curing time are some of the many variables that impact the CCS. For instances, porosity, which in turn affects strength and durability, is a direct result of the water-to-binder ratio in concrete. Likewise, the dimensions and classification of aggregates affect both the workability and ultimate strength of the mixture. Comprehending the interplay among these factors is essential for formulating concrete mixtures that satisfy particular performance standards.

ML models can identify the optimal combination of mixture components, improving strength outcomes while reducing costs and minimizing environmental impact. By incorporating supplementary cementitious materials like fly ash, ML-assisted design can help reduce the carbon footprint of concrete production, which is a major contributor to global CO_2 emissions. Enhanced mixture optimization through ML also reduces material waste and extends the lifespan of concrete structures, contributing to long-term sustainability.

Numerous studies have explored the application of ML in structural engineering, with a particular emphasis on forecasting the CCS. One study successfully employed the ANN algorithm to predict the CS of high-performance concrete (HPC) containing materials like silica nanoparticles and copper slag [33]. It demonstrates ANN's capability in managing the complex characteristics of these materials. Another investigation introduced four different ML models for predicting both the compressive and tensile strength of HPC [34]. The results highlighted that Gradient Boosting Regressor (GBR) and Extreme Gradient Boosting (XGBoost) produced the highest accuracy, underscoring these algorithms' effectiveness in detecting complex data patterns. Research on lightweight concrete (LWC) involved data collection from 120 samples to develop predictive models [35]. It reveals that the Support Vector Machine (SVM) model was the most effective in evaluating the CS of LWC, demonstrating its strength in handling diverse datasets. In order to gain a better understanding of how fibrous concrete behaves under different environments, heuristic regression methods have been used to forecast its strength and ultrasonic pulse velocity [36]. The Random Forest (RF) algorithm has been used to predict the CS of lightweight self-compacting concrete (LSC), involving an testing of eight key input parameters to better understand the factors influencing model accuracy [37]. Additionally, a model based on the GEP algorithm has been developed to estimate the CS of high-strength concrete (HSC) using an extensive dataset of 357 data points. This model demonstrated the robustness of GEP in handling complex material properties [38]. These studies collectively demonstrate the varied applications and efficacy of different machine learning models in forecasting concrete properties. Table 1 provides a comprehensive summary of the ML models and methodologies, offering a detailed overview of the progress made in this field.

Although numerous studies have explored the prediction of concrete's CS through various machine learning methods, there has been little to no exploration of classifying concrete according to these predicted strength values. While predicting the CS of concrete provides valuable insights into its structural performance, classifying concrete based on its predicted strength adds another dimension of utility. Classification enables the automatic categorization of concrete into predefined strength categories, such as high, medium, or low strength, which is crucial in practical applications. By leveraging machine learning for this classification, construction professionals can make more informed decisions regarding the appropriate use of concrete in various structural components. This not only improves the efficiency and safety of construction projects but also helps optimize the selection of materials based on strength requirements. Furthermore, machine learning-driven classification can reduce human error in interpreting complex data, streamline quality control processes, and enable real-time decision-making through automated systems. Ultimately, classifying CS using machine learning enhances the accuracy and applicability of predictions in real-world construction scenarios, allowing for more efficient use of resources and improved structural integrity.

Research novelty and contribution

This study specifically focuses on addressing the gap in the current body of research by exploring the classification of concrete based on its predicted compressive strength (CS). Unlike previous studies that primarily emphasize prediction, this work aims to investigate and develop a machine-learning framework for categorizing concrete into distinct strength classes. The subsequent delineates the authors' distinct contributions:

1. Provide varied machine learning models, known for their robustness and accuracy, to enhance the reliability of the classification process of CS of concrete.
2. Using Bayesian Optimization alongside k-fold cross-validation for parameter tuning substantially enhances generalization performance.
3. Additional analysis was conducted using confusion matrices and Receiver-Operating-Characteristic (ROC) curves to acquire a deeper understanding of the strengths

Table 1 ML models adopted in the past literature

Concrete type	Dataset size	Adopted ML models	Ref
OPC and HPC	2817	ANN, ANFIS	[39]
HPC	1030	Gradient boosting regression tree (GBRT)	[25]
	471	Ensemble, non-ensemble ML approach	[40]
	1030	XGBoost, RF, SVR	[41]
	270	GEP, DT, and Bagging	[42]
	1761	SVM	[43]
	2171	BiLSTM, CNN, GRU, LSTM	[44]
HSC	225	FA, RF	[45]
	1133	ANN	[46]
	357	GEP	[38]
	681	XGBR-BR, SVR-RFR, GBR-DTR	[47]
UHPC	78	ANN	[48]
	626–317	XGBoost, LightGBM, hybrid XGBoost-LightGBM	[49]
	810	AB, RF, GB	[50]
SCC	7	ANFIS	[51]
Pozzolanic concrete	310	GEP, ANN	[52]
RHA-based concrete	1404	LGB, XGBoost, RF, hybrid XGBoost-LGB, hybrid XGBoost-RF	[53]
	1212	CatBoost, GBM, CNN, GRU	[54]
	138	ANN, XGBoost, GBM	[55]

and weaknesses of the models in the classification process.

4. Implement SHAP analysis to interpret model predictions, providing insights into feature importance and the influence of key variables on compressive strength classification.
5. Introduce an interactive GUI to facilitate user-friendly access and interpretation of the classification results. The combination of advanced ML techniques and practical implementation tools represents a novel approach that could significantly improve the understanding and application of concrete strength classification in the construction industry.

Materials and methods

Database acquisition

The dataset used in this study was compiled from various reputable sources, ensuring a diverse and comprehensive collection of concrete compressive strength (CCS) data. A total of 1298 samples were gathered from Song et al. [19, 40], and Yeh [56]. These sources provide a rich set of experimental data, covering a wide range of concrete compositions, curing times, and testing conditions, which are essential for accurate machine learning modeling. Each data sample includes detailed information on the mix proportions of key ingredients, including cement, blast furnace slag, fly ash, water, superplasticizer, coarse aggregate, fine aggregate, along with the age of the concrete and the corresponding compressive strength. To facilitate analysis and classification, CS values in the collected dataset were categorized into three distinct ranges:

- Low compressive strength (2.33–25.01 MPa): This range typically represents early-age or lower-grade concrete mixes, often used in non-structural applications such as sidewalks, driveways, and certain residential elements.
- Medium compressive strength (25.02–55.02 MPa): Concrete in this range is widely used in standard structural applications, including residential and commercial buildings, bridges, and infrastructure projects requiring a balance of strength and workability.
- High compressive strength (55.03–85.70 MPa): This category includes high-performance concrete used in specialized applications such as high-rise buildings, long-span bridges, and structures subjected to extreme loads or environmental conditions.

For robust model training and evaluation, the dataset was divided into 80% for training and 20% for testing, a ratio that ensures effective learning while maintaining a sufficient portion of unseen data for performance assessment. The diversity of the dataset enhances the generalization and robustness of the machine learning models, making the classification results applicable to a broad spectrum of concrete types. By integrating data from multiple studies, the dataset captures the variability inherent in real-world concrete properties, thereby improving the reliability and applicability of the models developed in this research (Figs 1, 2).

Statistical summary

The descriptive statistics presented in Table 2 summarize the key characteristics of the dataset, including the minimum, maximum, mean, median, standard deviation, and count for each variable. The corresponding histograms in Fig. 3 provide a visual representation of the frequency distributions, offering insight into the spread and variation of the data. The cement content in the dataset ranges from 74.00 to 588.20 kg/m^3, with a mean value of 277.59 kg/m^3 and a median of 272.55 kg/m^3. The histogram shows a distribution slightly skewed towards lower values, with most samples having cement content around 250–400 kg/m^3. Blast furnace slag content varies from 0.00 to 359.40 kg/m^3, but the median value is zero, indicating that a large number of samples do not contain any slag. This is reflected in the histogram, which is highly skewed towards zero. Fly ash content follows a more spread-out distribution, ranging from 0.00 to 330.00 kg/m^3, with a mean of 71.79 kg/m^3 and a median of 79.00 kg/m^3.

Water content in the dataset has a relatively normal distribution, ranging from 121.75 to 255.00 kg/m^3, with a mean of 183.95 kg/m^3 and a median of 185.70 kg/m^3. Most samples have water content between 150 and 200 kg/m^3. The superplasticizer content varies significantly, with values between 0.00 and 32.20 kg/m^3, but the histogram indicates that many samples contain very low or zero amounts, resulting in a skewed distribution. The coarse aggregate content ranges from 436.00 to 1226.00 kg/m^3, with a mean of 949.29 kg/m^3 and a median of 958.10 kg/m^3. Its distribution is fairly normal, with most values clustering around 900–1000 kg/m^3. Fine aggregate content shows a wider variation, ranging from 30.00 to 1293.00 kg/m^3, with a mean of 777.63 kg/m^3 and a median of 780.09 kg/m^3. The histogram suggests a bimodal distribution, indicating two dominant ranges in the dataset.

The age of the samples varies widely, from 1 to 365 days, with a mean of 42.02 days and a median of 28.00 days. The histogram shows a strong peak at lower values, suggesting that most samples are tested at early ages, particularly around 1–28 days. The compressive strength of concrete ranges from 2.33 to 85.70 MPa, with a mean of 36.01 MPa and a median of 34.95 MPa. The distribution spans a broad range, but most samples fall within 25–55 MPa. The compressive

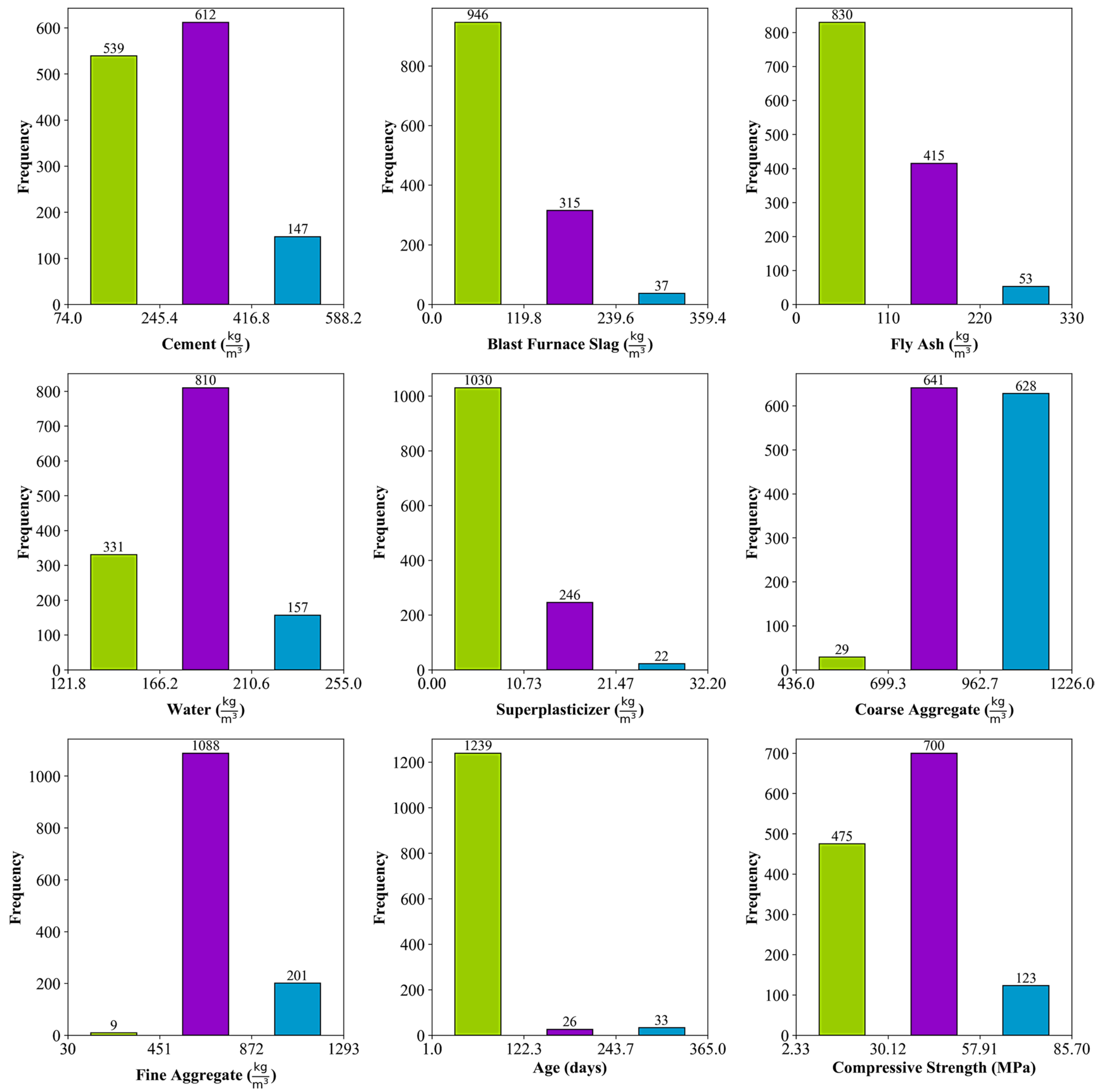

Fig. 1 Histograms of collected dataset

strength data is further classified into three categories: low strength (LSC), normal strength NSC), and high strength (HSC). Low-strength concrete has a compressive strength ranging from 2.33 to 25.02 MPa, with a mean of 16.93 MPa and a total count of 331 samples. Normal-strength concrete falls within the range of 25.08 to 55.02 MPa, with a mean of 38.03 MPa and the largest count of 804 samples. High-strength concrete ranges from 55.06 to 85.70 MPa, with a mean of 64.79 MPa and a smaller count of 163 samples. The histogram shows that the majority of the samples belong to the normal strength category, while fewer samples fall into the low and high-strength categories.

Overall, the dataset provides a diverse range of concrete mix compositions, with significant variations in binder materials such as cement, slag, and fly ash, as well as in aggregate and admixture content. The distribution patterns show that most samples contain low or no blast furnace slag and superplasticizer, while water, coarse, and fine aggregate contents follow relatively normal distributions. The compressive strength data is well-structured

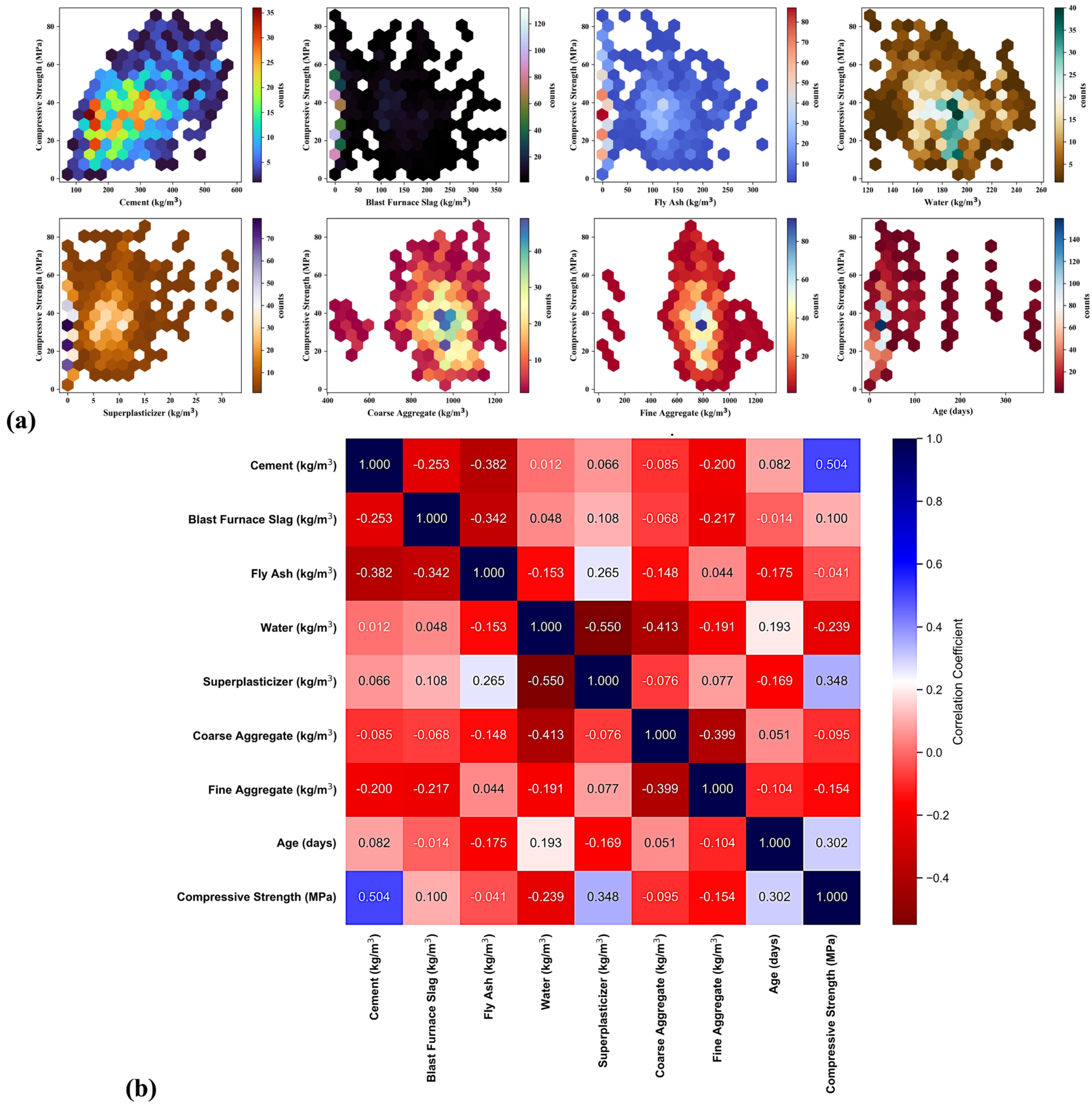

Fig. 2 Correlation analyses: **a** hexbin plots and **b** heatmap

into three distinct strength classes: low, normal, and high, allowing for effective classification. Additionally, the age distribution indicates that most tests are conducted at early ages, which could impact the classification of strength over time. From a machine learning perspective, the dataset appears well-suited for classification analysis. The presence of clearly defined strength classes provides a strong basis for supervised learning models to distinguish between different categories of compressive strength.

Correlation analysis

The correlation analysis presented in Fig. 2 consists of two parts: the correlation heatmap (Fig. 2a) and the hexbin plots (Fig. 2b). These visualizations provide a deeper

Table 2 Descriptive statistics of the database collected

Input	Variable	Unit	Min	Max	Mean	Median	Std. Deviation	Count
Cement	X1	kg/m^3	74.00	588.20	277.59	272.55	102.77	1298
BF Slag	X2	kg/m^3	0.00	359.40	64.83	0.00	82.50	1298
Fly ash	X3	kg/m^3	0.00	330.00	71.79	79.00	74.63	1298
Water	X4	kg/m^3	121.75	255.00	183.95	185.70	24.02	1298
SP	X5	kg/m^3	0.00	32.20	5.98	6.00	5.69	1298
Cagg	X6	kg/m^3	436.00	1226.00	949.29	958.10	110.41	1298
Fagg	X7	kg/m^3	30.00	1293.00	777.63	780.09	116.04	1298
Age	X8	Days	1.00	365.00	42.02	28.00	56.72	1298
CS	Y	MPa	2.33	85.70	36.01	34.95	15.90	1298
Low	LSC	MPa	2.33	25.01	16.93	17.37	5.52	331
Normal	NSC	MPa	25.02	55.02	38.03	37.41	7.80	804
High	HSC	MPa	55.03	85.70	64.79	63.40	7.87	163

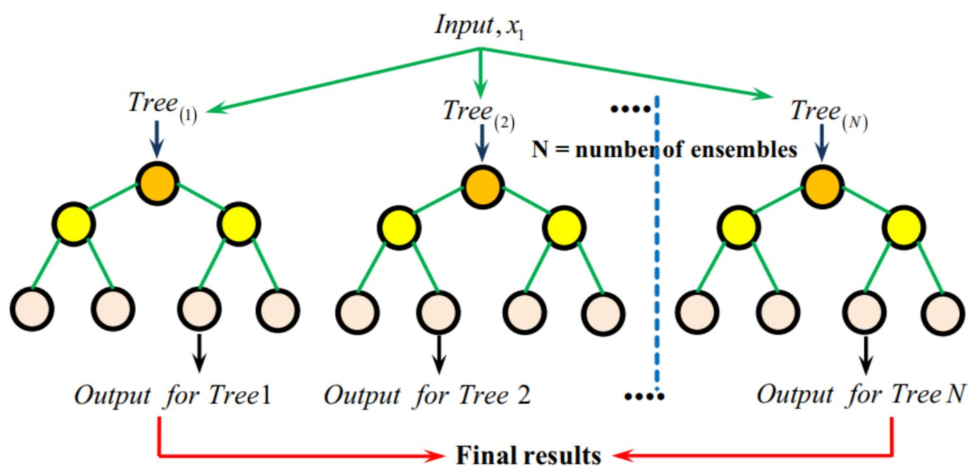

Fig. 3 Framework for RF model [59]

understanding of the relationships between input variables and their influence on compressive strength. The correlation heatmap displays the Pearson correlation coefficients between different variables in the dataset. The strength and direction of correlations are represented by color intensity, where blue indicates a strong positive correlation, and red indicates a strong negative correlation. From the heatmap (Fig. 2a), cement content shows the highest positive correlation (0.504) with compressive strength, suggesting that increasing cement content generally leads to stronger concrete. Superplasticizer content and age also exhibit a moderate positive correlation (0.348 and 0.302, respectively) with compressive strength, indicating that these factors contribute positively to strength development. Conversely, water content shows a negative correlation (– 0.239), implying that higher water content reduces compressive strength, likely due to increased porosity. Fly ash and blast furnace slag have weak or negative correlations, indicating that their effect on compressive strength is less direct or dependent on other factors such as curing conditions and age.

The hexbin plots provide a density-based representation of the relationship between each independent variable and compressive strength. In the plot (Fig. 2b) for cement content, a clear positive trend is visible, reinforcing the strong correlation observed in the heatmap. The blast furnace slag and fly ash plots show scattered distributions with no strong pattern, aligning with their weak correlations. The water content plot shows a denser region around 150–200 kg/m^3, where most samples cluster, with a slight negative trend in strength, supporting the negative correlation. The superplasticizer plot shows higher compressive strengths at lower superplasticizer levels, but the relationship appears less linear, possibly due to interactions with other mix components. The coarse and fine aggregate plots show widely distributed values, suggesting weaker direct correlations. Finally, the age plot exhibits discrete clusters, indicating that samples were tested at specific ages rather than continuous distribution. Overall, the correlation heatmap confirms the primary influence of cement, water, and age on compressive strength, while the hexbin plots visually reinforce these relationships. The dataset demonstrates some clear trends that are useful for classification, particularly in distinguishing low-, normal-, and high-strength concrete based on key mix proportions. However, certain variables exhibit weak correlations, suggesting that additional factors, such as curing conditions or interactions between components, may also play a role in determining compressive strength.

Description of ML models

RF model

The Random Forest (RF) model is an advanced regression method designed to enhance predictive accuracy by combining multiple decision trees. Developed as a powerful ensemble learning technique, RF is recognized for its efficiency and flexibility in establishing relationships between input and output parameters [57]. The model operates by constructing several independent decision trees from subsets of the original dataset, generating a more reliable and accurate prediction through collective decision-making. The construction of the RF model involves three key phases. **First**, regression trees are built using training data generated through bootstrap sampling, where approximately two-thirds of the original data is randomly selected, and the remaining one-third is excluded to form out-of-bag (OOB) data. **Second**, the outputs from all the decision trees are averaged to create a unified prediction model. Finally, the model's performance is validated using the OOB data, which serves as an independent benchmark for accuracy assessment.

A critical aspect of the RF model is its use of OOB data for validation, which enhances predictive reliability and distinguishes RF from other ensemble models. In tree construction, optimal branching attributes are randomly chosen from a predetermined set, which encourages diversity among the trees and improves overall predictive performance [58]. After the training phase is complete, the final class of a new sample is determined by majority voting across all the trees, with the aggregated outcome calculated based on a predefined equation. The accuracy of each individual tree is also evaluated to measure its contribution to the overall model performance. The RF model's architecture is depicted in Fig. 3.

AdaBoost model

AdaBoost is an ensemble-based ML algorithm widely recognized as an advanced boosting technique applicable to both classification and regression tasks [60]. In particular, AdaBoost regression is designed to address regression problems. Similar to other ensemble methods, AdaBoost regression enhances predictive accuracy by combining multiple simple models to form a stronger and extra reliable predictor. The fundamental principle behind AdaBoost regression is to iteratively build a strong model by integrating multiple weak models, where each subsequent model assigns greater emphasis to the data points that previous models misclassified. With each iteration of training, AdaBoost regression highlights data points that were previously mis-predicted, allowing weak models to be fine-tuned on a weighted dataset version. This process keeps going until either all of the models are trained or the accuracy level is achieved. Figure 4 illustrates the AdaBoost model architecture.

XGBoost model

XGBoost is a cutting-edge ML library known for its exceptional capability to handle great datasets and solve complex predictive modeling problems. It operates using a gradient boosting framework, where models are built sequentially, with each new model aiming to minimize the errors produced by the preceding models [62]. This iterative correction mechanism enhances the model's overall accuracy and performance.

One of the standout features of XGBoost is its capacity to efficiently manage missing data, ensuring that incomplete datasets do not compromise predictive performance. It also integrates regularization techniques such as Lasso and Ridge penalties, which help prevent overfitting and improve model generalization [63]. Furthermore, XGBoost leverages parallel processing, significantly reducing computation time and enhancing scalability. Its ability to produce precise and scalable models makes it particularly valuable in scenarios where predictive accuracy and efficiency are critical. As a result, XGBoost has emerged as a reliable framework for data scientists and ML engineers seeking to develop robust predictive models that can direct informed business decisions and optimize results. The architecture of the XGBoost model is shown in Fig. 5.

LightGBM model

LightGBM is a fast and highly efficient gradient-boosting framework designed for large-scale machine learning tasks [64]. Its development aimed to outperform other popular gradient-boosting libraries, such as XGBoost, by enhancing processing speed and scalability without compromising performance. This framework is particularly effective when managing large datasets and handling complex models, making it suitable for a wide range of tasks, including classification, regression, and ranking. Figure 6 illustrates the architecture of the LightGBM model.

A key innovation in LightGBM is its histogram-based learning algorithm. Instead of working directly with

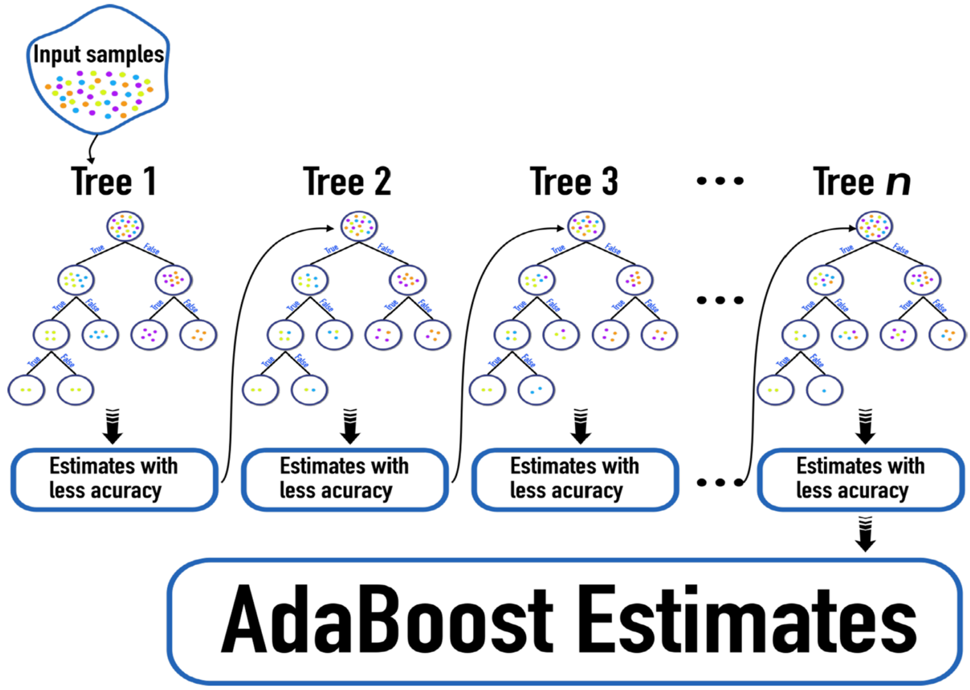

Fig. 4 Architecture of AdaBoost model [61]

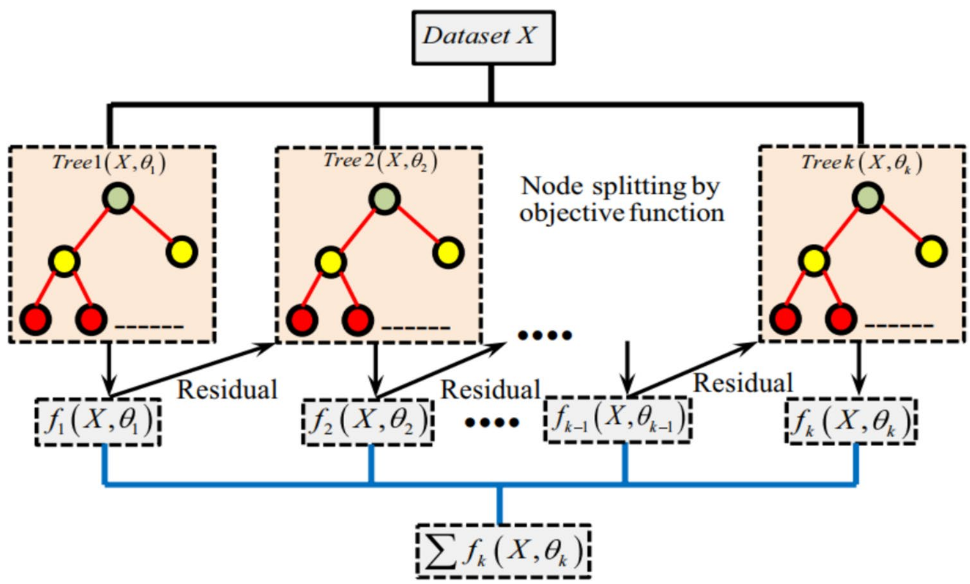

Fig. 5 Architecture of XGBoost model [59]

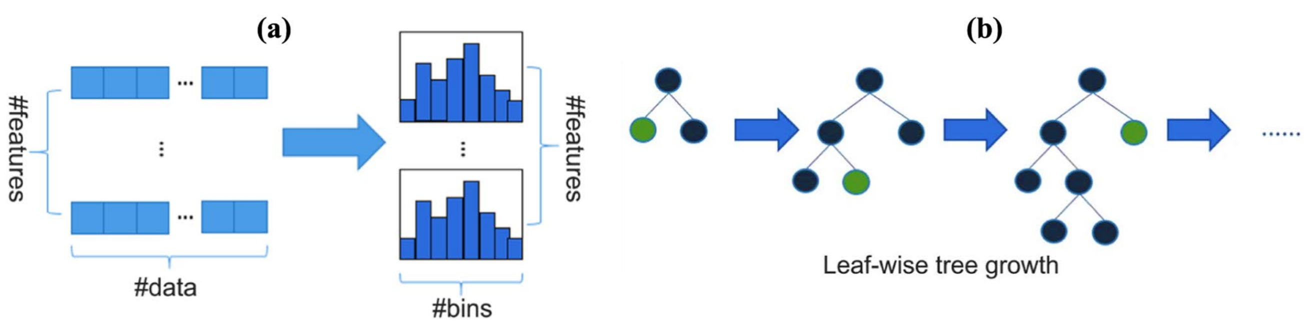

Fig. 6 Architectures of LightGBM model: **a** Histogram process and **b** Leaf-wise leaf growth process [65]

floating-point values, LightGBM converts them into a set of discrete integer values, represented by "$\boldsymbol{k}$". This transformation allows the construction of a histogram with a fixed width of k. During data traversal, the framework gathers statistical information based on the discretized values, which serve as histogram indices. Once the data traversal phase is complete, the histogram retains essential statistics that are then used to determine the optimal split point based on the discrete histogram values. This approach improves computational efficiency by reducing memory usage and speeding up the learning process. This process is depicted in Fig. 6a.

Traditional decision tree growth strategies generally utilize a Level-wise approach, which is inefficient because it treats all leaves at the same level equally, resulting in unnecessary memory usage. In contrast, LightGBM adopts a more efficient Leaf-wise growth strategy. This method involves identifying the leaf with the highest split gain among all leaves, splitting it, and repeating the procedure. As a result, the Leaf-wise strategy can reduce errors and achieve greater precision with the same number of splits as the level-wise method. However, a potential drawback of the Leaf-wise approach is the increased risk of creating a deeper decision tree, which may lead to overfitting. To address this issue, LightGBM enforces a maximum depth limit on Leaf-wise growth, thus mitigating overfitting while maintaining high efficiency. The Leaf-wise growth process is illustrated in Fig. 6b.

CatBoost model

CatBoost is an ML strategy that was created to efficiently process categorical features while delivering high-accuracy predictions [66]. Unlike more conventional preprocessing methods like one-hot encoding or label encoding, this variation of gradient boosting can handle numerical and categorical data [67]. Instead, CatBoost uses ordered boosting, a unique technique that reduces overfitting by encoding categorical variables dynamically during training. Unlike conventional approaches, ordered boosting processes data in a permutation-driven manner, ensuring that each observation is trained without being influenced by future data points [68, 69]. This approach improves model robustness and predictive accuracy, especially when dealing with structured datasets with numerous categorical variables. CatBoost is particularly well-suited for both classification and regression tasks, excelling in scenarios where categorical features play a significant role [70–72]. Additionally, the algorithm provides feature importance rankings, which help in feature selection and offer insights into how different variables impact predictions. The schematic of the CatBoost model is illustrated in Fig. 7.

Hyperparameters tuning

Hyperparameter tuning is a critical step in optimizing machine learning models, and Bayesian Optimization (BO) offers an advanced and efficient approach [74]. Unlike conventional grid search or random search methods, which explore hyperparameters without direction, BO leverages probabilistic models to intelligently search for the best configurations. The process starts with an initial set of hyperparameters, whose performance is evaluated using a selected metric [75–81]. BO then updates its probabilistic model to predict which hyperparameter combinations are most promising. It balances exploration (searching new regions of the hyperparameter space) and exploitation (refining known high-performing areas) to efficiently converge on an optimal solution, reducing computational cost and improving model performance.

To ensure robust generalization, stratified five-fold cross-validation (CV) is integrated with BO. Unlike standard cross-validation, stratified CV maintains the same proportion of class labels in each fold, preventing data imbalance from affecting model evaluation. The choice of five-fold cross-validation over the more commonly used ten-fold cross-validation was based on a balance between computational efficiency and model reliability. Five-fold cross-validation provides a robust estimate of model performance while significantly reducing computational cost compared to ten-fold cross-validation, which is particularly relevant when working with large datasets and complex models. Additionally, previous studies have demonstrated that five-fold cross-validation offers comparable performance estimates to ten-fold cross-validation while requiring fewer iterations, making it a practical choice for this study. To ensure the reliability of the evaluation, the dataset was randomly shuffled before splitting, minimizing potential biases in the training and validation sets. The average accuracy across the five folds serves as the fitness value in the BO process, ensuring that the selected hyperparameters optimize performance across all models while preventing overfitting [82–87].

Assessment criteria

Evaluating the effectiveness of each model is vital for ensuring the practicality and scientific validity of the results [88]. Training datasets are utilized to construct models, yet they simply indicate the extent to which a model corresponds with the supplied data. Validating these models requires testing datasets, which measure their ability to generalize to novel, unseen data. It is crucial to validate the model using testing datasets to ensure its reliability and practicality in real-world scenarios, as it is difficult to identify if a model is

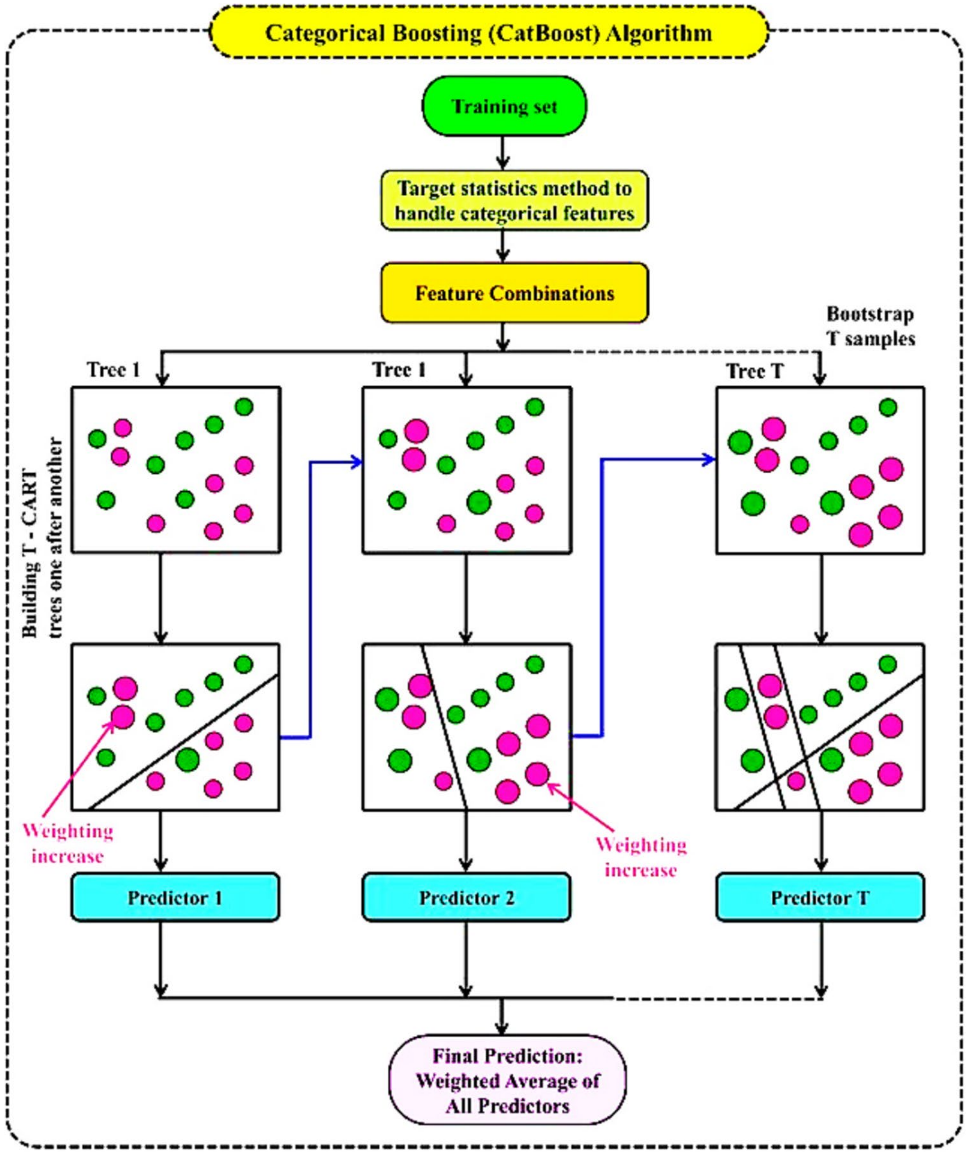

Fig. 7 Architecture of CatBoost model [73]

just learning the training data or can accurately predict new inputs without them [89].

Models are compared and evaluated in this research through both visual and quantitative evaluation tools [73, 74]. Visual evaluation utilize graphical tools (like confusion matrices), to offer intuitive insights into model performance, which can help uncover patterns and anomalies that numerical metrics alone may not reveal [71, 72]. On the other hand, quantitative assessments provide numerical metrics for an objective evaluation of performance, which are necessary for comparing various models and identifying the best one. Together, these approaches ensure that the model is both practical and scientifically sound for real-world applications [90].

The metrics selected for evaluation include Accuracy, Precision, Recall, F1 Score, and Area Under the Curve (AUC). Accuracy is defined as the ratio of correctly predicted instances to the total instances, serving as a straightforward measure of performance. However, it may be misleading in cases of class imbalance. The F1 Score represents the harmonic mean of Precision and Recall, making it particularly useful when dealing with uneven class distributions. Precision indicates the ratio of correctly predicted positive observations to the total predicted positives, signifying a low false positive rate when high. Recall (or Sensitivity) measures the ratio of correctly predicted positive observations to all actual positives, reflecting a low false negative rate when high. AUC quantifies the area under the Receiver Operating Characteristic (ROC) curve, assessing a classifier's ability to differentiate between classes. A higher AUC signifies better model performance, with values ranging from

0 (indicating random guessing) to 1 (indicating perfect discrimination).

These metrics were chosen because they provide a comprehensive evaluation of model performance, addressing various factors, including overall accuracy, the balance between false positives and false negatives, and the model's capability to distinguish between different classes. By employing these metrics, the evaluation and comparison of various classification models can be conducted thoroughly, ensuring the selected model is both accurate and reliable for practical use. The equations for the selected classification metrics are detailed in Table 3.

Feature interpretability and importance

SHapley Additive exPlanations (SHAP) analysis is a powerful interpretability tool used to quantify the impact of individual features on machine learning model predictions [71, 91]. It is based on cooperative game theory and assigns a SHAP value to each feature, indicating its contribution to a specific prediction. The SHAP framework ensures that the sum of feature contributions matches the model's output, providing a consistent and explainable breakdown of feature importance [92]. In this study, SHAP analysis was applied to interpret the predictions of machine learning models classifying concrete compressive strength into Low, Normal, and High categories. The SHAP summary dot plots were used to visualize feature importance across different strength classes. Each point on the plot represents a SHAP value for a particular feature in a given instance, with color gradients reflecting feature magnitude. Positive SHAP values indicate a positive effect on compressive strength, whereas negative values denote a reduction in strength.

Graphical user interface

To make the predictive model for estimating the CS more accessible and user-friendly, a graphical user interface (GUI) was developed using Python's Tkinter library. As a standard GUI toolkit, Tkinter provides a straightforward method for building interactive applications [93]. Setting up the Python environment was the first step in the development process. Python comes with Tkinter pre-installed. Input fields allow users to input necessary features, a button initiates the prediction process, and labels or instructions facilitate navigation; the design of the interface prioritizes simplicity and user guidance. The main features of the user interface involves TextEntry widgets for input, a Button widget to activate the prediction, and Label or Text widgets to display the resulting CCS value. The trained predictive model is incorporated into the GUI by using a library like pickle, which allows the model to be loaded and generate predictions based on the inputs provided by the user. The GUI app is available on the open-source software hosting platform GitHub (https://github.com), benefiting from features like version control, collaboration tools, and simple distribution.

Table 3 Equations of the chosen classification metrics

Metric	Equation
Accuracy	$\frac{\text{True Positives}+\text{True Negatives}}{\text{True Positives}+\text{True Negatives}+\text{ False Positives}+\text{True Negatives}}$
Precision	$\frac{\text{True Positives}}{\text{True Positives}+\text{False Positives}}$
Recall	$\frac{\text{True Positives}}{\text{True Positives}+\text{False Negatives}}$
F1 Score	$2\times\frac{\text{Precision}\times\text{Recall}}{\text{Precision}+\text{Recall}}$
AUC	$\sum_{i=1}^{n-1}\frac{(FPR_{i+1}-FPR_i)\times(TPR_i+TPR_{i+1})}{2}$

Where TPR is the True Positive Rate at different threshold levels; FPR is the False Positive Rate at different threshold levels; n is the number of thresholds

Results and discussion

Iterations versus mean test score

Figure 8 depicts the Bayesian optimization (BO) procedure for the adopted models. Each plot shows the average test score (based on the accuracy metric) on the y-axis against the number of iterations on the x-axis. Lower scores represent better model performance. In the RF optimization (Fig. 8a), there are significant fluctuations in scores, with notable peaks and valleys across the iterations. The algorithm begins with sharp variabilities, indicating a broad exploration of hyperparameters. As iterations progress, the fluctuations gradually reduce, indicating a process of convergence, but the trajectory remains volatile, suggesting ongoing refinement or challenges in hyperparameter stability. The AdaBoost optimization (Fig. 8b) displays substantial variability in test scores throughout the optimization process. The early iterations show significant dips, followed by a gradual increase and leveling off of scores, though variability persists even at later stages. The exploration of suboptimal configurations is evident, with sudden spikes and dips reflecting attempts to refine the model. The latter part of the process suggests some level of stabilization. The XGBoost optimization (Fig. 8c) demonstrates extreme fluctuations in mean test scores during the early stages, followed by gradual convergence. Initial iterations exhibit significant spikes, which are indicative of the exploration of a wide hyperparameter space. As the algorithm proceeds, the test scores stabilize with fewer extreme dips, suggesting successful refinement and tuning of hyperparameters. The LightGBM optimization (Fig. 8d) shows a pattern of initial high variance in scores, with

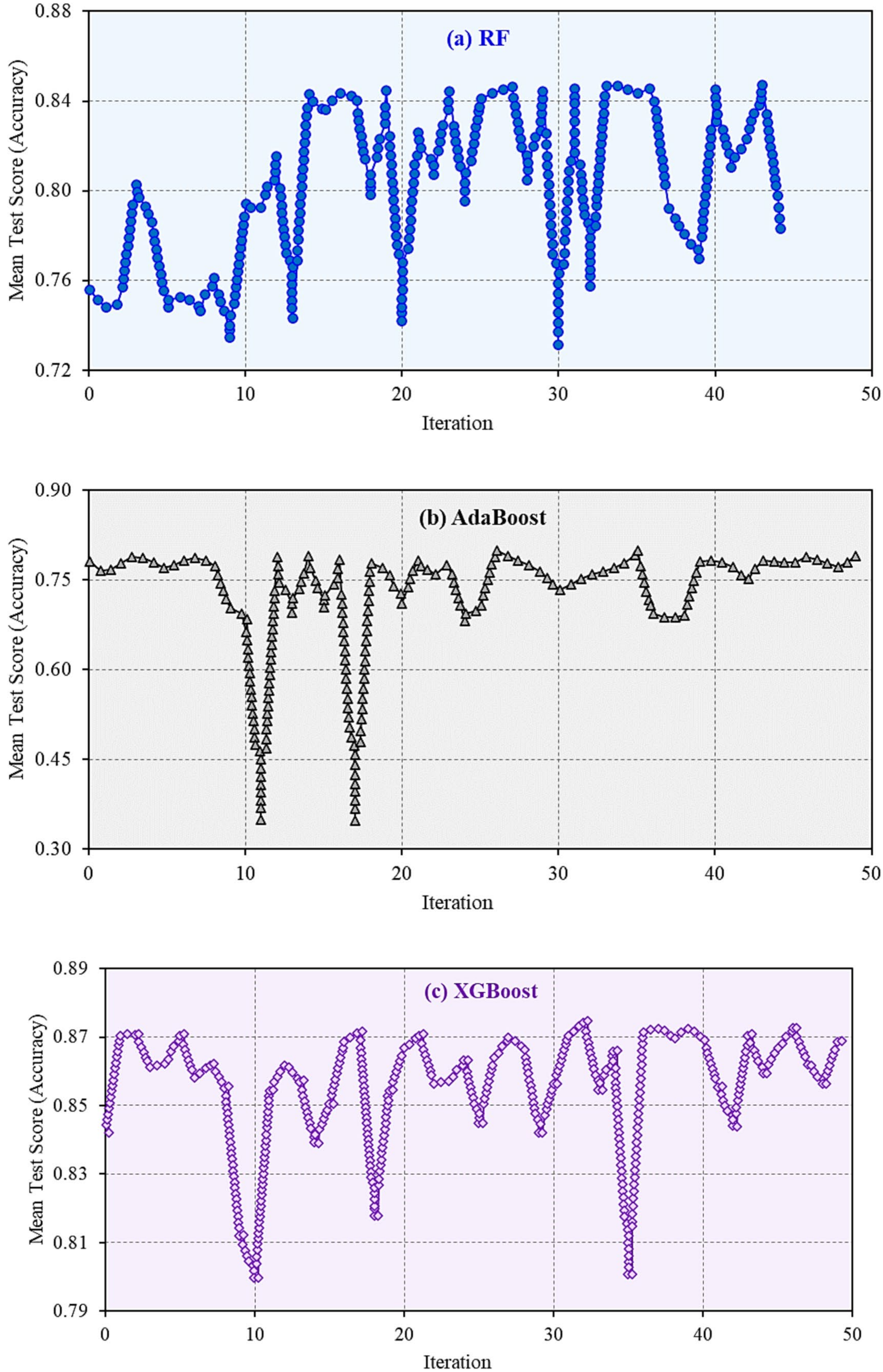

Fig. 8 Classification performance of **a** RF, **b** AdaBoost, **c** XGBoost, **d** LightGBM, and **e** CatBoost models during BO process based on mean accuracy

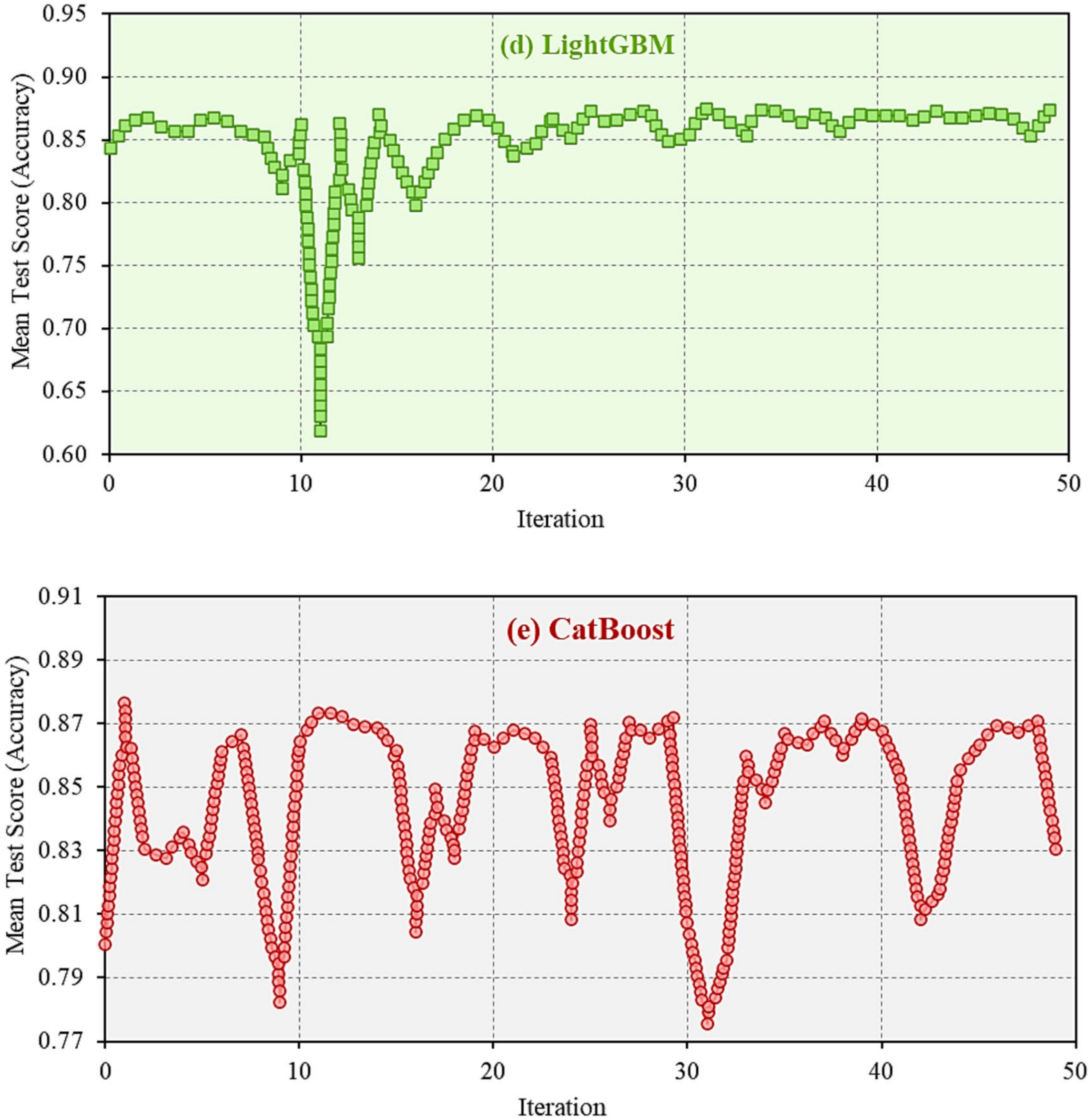

Fig. 8 (continued)

drops and subsequent recoveries. As iterations progress, the algorithm exhibits a convergence pattern where the scores begin to stabilize. The CatBoost (Fig. 8e) exhibits a unique pattern of optimization, with noticeable oscillations in the early stages. The model begins with substantial fluctuations as it explores the hyperparameter space. Over time, the mean test scores display a slight stabilization, with periodic dips and recoveries.

Optimal hyperparameters

Table 4 presents the optimal hyperparameters identified through the BO tuning process for the adopted models: RF, AdaBoost, XGBoost, LightGBM, and CatBoost. These hyperparameters have been fine-tuned to achieve the best possible performance for each model. The optimal hyperparameters for the RF model include a high number of estimators (600), which indicates that the model uses a large number of trees to enhance predictive accuracy. With a maximum depth of 16, the RF model can capture more intricate patterns in the data as the trees grow relatively deep. An additional minimum sample split of (2) and a minimum sample leaf of (1), allows for splitting the nodes until very small leaf nodes are formed, ensuring a finer granularity in predictions and potentially increasing the model's ability to fit closely to the data. For the AdaBoost model, the optimal hyperparameters include (10) estimators, which is a relatively low number, focusing more on the iterative boosting process than on the number of weak learners. The learning rate is set to be 1.166, suggesting that the Adaboost

Table 4 Optimal hyperparameters derived from the Bayesian optimization tuning process for each ML model

Model→ Hyperparameter↓	RF	AdaBoost	XGBoost	LightGBM	CatBoost
n_estimators	600	10	600	364	–
max_ depth	16	–	13	30	9
min_child_weight	–	–	7	1	–
learning_rate	–	1.166	0.3	0.0617	0.254
min_samples_split	2	–	–	–	–
min_samples_leaf	1	–	–	–	–
l2_leaf_reg	–	–	–	–	4

model updates the weights quickly, speeding up learning but increasing the risk of overfitting. The model uses the "linear" loss function, which helps maintain a balance between correctly predicting most of the data and focusing on harder-to-predict instances.

The XGBoost model is optimized with (600) estimators, which balances performance and efficiency by using a relatively high number of trees. The learning rate is set at 0.3, which enables the model to learn more gradually, avoiding drastic updates that could lead to suboptimal solutions. The maximum depth is set to 13, allowing the trees to grow deep enabling the model to capture complex interactions within the data. The model is further regularized by setting the `min_child_weight` to 7, which prevents overfitting by requiring a minimum sum of instance weights in the child nodes. For the LightGBM model, the optimal hyperparameters include (364) estimators and a learning rate of 0.0617. This relatively slow learning rate allows the model to update gradually, reducing the likelihood of overshooting the optimal solution. The maximum depth is set to 30, indicating that the trees are allowed to grow very deep, enabling the model to capture complex patterns in the data. The minimum child weight is set to 1.0, ensuring that the model enforces constraints on splitting nodes with smaller sums of instance weights, which helps regularize the model and reduce overfitting. Finally, the optimal learning rate for the CatBoost model was 0.254, which is moderate and allows the model to learn effectively while minimizing the risk of overshooting the optimal solution. The depth is set to 9, enabling the trees to grow relatively deep capturing complex data patterns. The `l2_leaf_reg` parameter is set to 4, providing regularization that controls the complexity of the trees by penalizing large leaf values, which helps prevent the model from overfitting.

Cross-validation analysis

The performance of the five adopted ML models across five cross-validation folds is displayed in Table 5 and Fig. 9. Lower accuracy values indicate a higher rate of misclassification, while higher values signify better model performance

Table 5 Accuracy values of developed models in BO + 5CV process

Fold	RF	AdaBoost	XGBoost	LightGBM	CatBoost
Fold 1	0.851	0.837	0.861	0.870	**0.885**
Fold 2	0.856	0.764	**0.875**	0.861	**0.875**
Fold 3	**0.861**	0.774	0.846	**0.861**	0.856
Fold 4	0.841	0.778	0.845	0.859	**0.865**
Fold 5	0.913	0.855	0.908	**0.918**	0.908

The bold values indicated the highest value in each fold

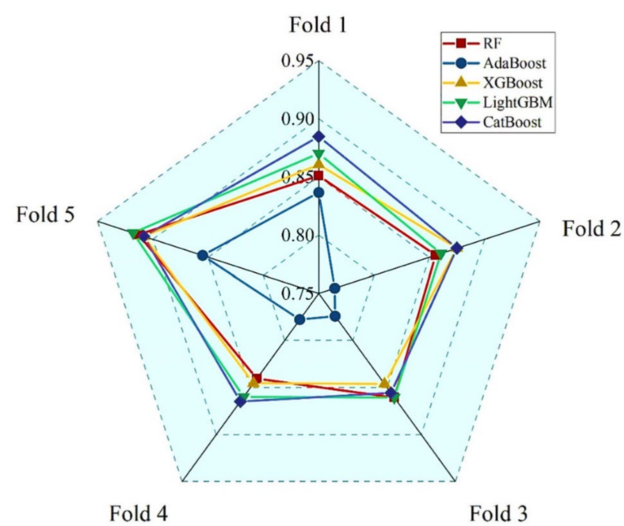

Fig. 9 Performance models during BO + 5CV process based on classification accuracy

in correctly predicting concrete CS categories. In Fold 1, CatBoost delivers the strongest performance with an accuracy of 0.885, demonstrating superior predictive capability right from the start. LightGBM closely follows with an accuracy of 0.870, showing good generalization. XGBoost also performs well with an accuracy of 0.861 but slightly lags behind the top two. RF maintains competitive performance at 0.851, while AdaBoost struggles the most, scoring 0.837. In Fold 2, XGBoost and CatBoost tie for the highest accuracy at 0.875, showcasing their ability to generalize effectively. LightGBM remains competitive with an accuracy of 0.861, although it is outperformed by the leading models. RF also performs well with an accuracy of 0.856, but AdaBoost records the weakest performance again with 0.764, marking a significant drop.

Conversely, both RF and LightGBM both achieved the top accuracy of 0.861 in Fold 3, demonstrating strong performance. XGBoost follows closely at 0.846, still showing solid results but falling short of the best. CatBoost performs well with an accuracy of 0.856 but does not reach the top. AdaBoost continues to struggle, with the weakest accuracy of 0.774. In Fold 4, CatBoost shines in this fold with the highest accuracy of 0.865, reflecting its strong generalization. LightGBM also performs well with an accuracy of 0.859, maintaining its competitive edge. XGBoost delivers solid performance with an accuracy of 0.845, while RF falls behind with an accuracy of 0.841. AdaBoost again lags behind the rest, with the weakest accuracy of 0.778. Lastly, LightGBM takes the lead in Fold 5 with the highest accuracy of 0.918, showcasing its strong generalization across this data split. CatBoost and XGBoost perform well with identical accuracies of 0.908, indicating their consistency and reliability. RF delivers a strong performance with an accuracy of 0.913, placing it among the top models in this fold. AdaBoost improves to 0.855 but still falls behind the other models.

Across all folds, The CatBoost, LightGBM, and XGBoost models consistently deliver the best performance, with CatBoost and LightGBM often achieving the highest accuracy values. These models show strong generalization capability and maintain high accuracy across the different data splits. The RF model performs well, particularly in Folds 3 and 5, demonstrating that it is a solid choice but is generally outperformed by the gradient-boosting models. The AdaBoost model, on the other hand, consistently exhibits lower accuracy-values across the folds, indicating that it may not be the best fit for this dataset.

Table 6 RF model performance before and after the tuning process

Class	Metric	Initial TR	Tuned TR	Initial TS	Tuned TS
LSC	Accuracy	0.866	0.998	0.854	0.927
	Precision	0.882	0.998	0.871	0.926
	Recall	0.866	0.998	0.854	0.927
	F1 Score	0.849	0.998	0.831	0.925
	AUC	0.950	1.000	0.923	0.964
NSC	Accuracy	0.771	0.999	0.746	0.850
	Precision	0.804	0.999	0.786	0.850
	Recall	0.771	0.999	0.746	0.850
	F1 Score	0.747	0.999	0.711	0.847
	AUC	0.867	1.000	0.798	0.924
HSC	Accuracy	0.911	0.999	0.892	0.938
	Precision	0.917	0.999	0.904	0.935
	Recall	0.911	0.999	0.892	0.938
	F1 Score	0.890	0.999	0.856	0.935
	AUC	0.961	1.000	0.934	0.971

Table 7 AdaBoost model performance before and after tuning process

Class	Metric	Initial TR	Tuned TR	Initial TS	Tuned TS
LSC	Accuracy	0.850	0.889	0.858	0.885
	Precision	0.844	0.891	0.852	0.882
	Recall	0.850	0.889	0.858	0.885
	F1 Score	0.845	0.890	0.852	0.883
	AUC	0.924	0.939	0.923	0.926
NSC	Accuracy	0.795	0.814	0.792	0.823
	Precision	0.793	0.813	0.791	0.822
	Recall	0.795	0.814	0.792	0.823
	F1 Score	0.791	0.814	0.786	0.819
	AUC	0.554	0.794	0.510	0.779
HSC	Accuracy	0.943	0.923	0.935	0.931
	Precision	0.942	0.919	0.931	0.926
	Recall	0.943	0.923	0.935	0.931
	F1 Score	0.942	0.920	0.931	0.925
	AUC	0.972	0.951	0.957	0.978

Model performance before and after hyperparameter tuning

Tables 6, 7, 8, 9, 10 listed the improvement in the classification performance of each developed model after hyperparameter tuning. These improvements are evaluated across five metrics: Accuracy, Precision, Recall, F1 Score, and AUC, during both training (TR) and testing (TS) stages, across three classes: LSC, NCS, and HSC. For LSC, LightGBM emerges as the top performer after tuning, showing excellent generalization across both training and testing sets. It reaches near-perfect Precision, Recall, and F1 scores of 0.998 in training, and maintains strong results in testing with Precision and Recall both improving to 0.930, and an F1 score of 0.930. XGBoost follows closely with improvements in both training and testing, with Precision and Recall reaching 0.996 during training, and testing metrics improving to 0.930 for Precision and 0.931 for Recall. However, it shows signs of slight overfitting, as reflected in its AUC drop from 1.000 (TR) to 0.966 (TS). CatBoost also shows strong generalization, with testing Precision improving to 0.922 and Recall to 0.923, alongside an F1 score of 0.922. RF performs well after tuning, showing notable improvements in testing Precision (from 0.871 to 0.926) and Recall (from 0.854 to 0.927), making it a solid option, although slightly less effective than LightGBM and XGBoost. AdaBoost, while showing some improvements, lags behind the other models with

Table 8 XGBoost model performance before and after tuning process

Class	Metric	Initial TR	Tuned TR	Initial TS	Tuned TS
LSC	Accuracy	0.963	0.996	0.919	0.931
	Precision	0.963	0.996	0.919	0.930
	Recall	0.963	0.996	0.919	0.931
	F1 Score	0.963	0.996	0.917	0.929
	AUC	0.992	1.000	0.975	0.966
NSC	Accuracy	0.942	0.994	0.862	0.873
	Precision	0.942	0.994	0.863	0.875
	Recall	0.942	0.994	0.862	0.873
	F1 Score	0.942	0.994	0.859	0.871
	AUC	0.989	1.000	0.936	0.930
HSC	Accuracy	0.977	0.996	0.942	0.942
	Precision	0.977	0.996	0.940	0.939
	Recall	0.977	0.996	0.942	0.942
	F1 Score	0.977	0.996	0.940	0.939
	AUC	0.997	1.000	0.980	0.976

Table 9 LightGBM model performance before and after tuning process

Class	Metric	Initial TR	Tuned TR	Initial TS	Tuned TS
LSC	Accuracy	0.971	0.998	0.919	0.931
	Precision	0.971	0.998	0.919	0.930
	Recall	0.971	0.998	0.919	0.931
	F1 Score	0.971	0.998	0.917	0.930
	AUC	0.995	1.000	0.965	0.967
NSC	Accuracy	0.958	0.999	0.865	0.865
	Precision	0.958	0.999	0.867	0.865
	Recall	0.958	0.999	0.865	0.865
	F1 Score	0.958	0.999	0.862	0.864
	AUC	0.994	1.000	0.923	0.938
HSC	Accuracy	0.985	0.999	0.946	0.935
	Precision	0.985	0.999	0.944	0.931
	Recall	0.985	0.999	0.946	0.935
	F1 Score	0.985	0.999	0.944	0.932
	AUC	0.999	1.000	0.977	0.981

Table 10 CatBoost model performance before and after tuning process

Class	Metric	Initial TR	Tuned TR	Initial TS	Tuned TS
LSC	Accuracy	0.901	0.988	0.881	0.923
	Precision	0.906	0.988	0.889	0.922
	Recall	0.901	0.988	0.881	0.923
	F1 Score	0.894	0.988	0.869	0.921
	AUC	0.964	0.999	0.951	0.956
NSC	Accuracy	0.829	0.987	0.812	0.873
	Precision	0.845	0.987	0.835	0.877
	Recall	0.829	0.987	0.812	0.873
	F1 Score	0.819	0.986	0.797	0.870
	AUC	0.921	0.999	0.874	0.904
HSC	Accuracy	0.928	0.996	0.931	0.950
	Precision	0.924	0.996	0.929	0.949
	Recall	0.928	0.996	0.931	0.950
	F1 Score	0.920	0.996	0.922	0.947
	AUC	0.976	1.000	0.966	0.964

a testing Precision of 0.882 and Recall of 0.885, making it the least effective model for LSC.

For NSC, CatBoost takes the lead, demonstrating strong overall performance across all metrics. After tuning, CatBoost achieves a testing Precision of 0.877, a Recall of 0.873, and an F1 score of 0.870, with an improved AUC of 0.904, reflecting its excellent ability to generalize. LightGBM also performs well, particularly in the training set where Precision, Recall, and F1 Score all reach 0.999, and the AUC hits 1.000. However, it shows moderate gains in testing, with Precision and Recall stabilizing at 0.865 and an F1 score of 0.864, which suggests good but not exceptional generalization. XGBoost performs similarly well in training with near-perfect metrics, but in testing, its performance slightly drops, showing Precision at 0.875, Recall at 0.873, and an F1 score of 0.871. RF improves significantly after tuning, with testing Precision increasing from 0.786 to 0.850, and Recall from 0.746 to 0.850, reflecting a balanced performance across metrics. AdaBoost, while improving marginally, still falls short compared to the other models, with a testing F1 score of 0.819 and an AUC of 0.779, indicating that it struggles to generalize effectively for NSC.

For HSC, CatBoost again shows strong performance after tuning, with near-perfect training Precision, Recall, and F1 scores of 0.996, and a solid testing performance with Precision at 0.949, Recall at 0.950, and an F1 score of 0.947. XGBoost follows closely, showing consistent performance with testing Precision steady at 0.940 and Recall at 0.942, but a slight drop in AUC from 0.980 to 0.976 reflects some minor overfitting. LightGBM performs well during training, with all metrics reaching 0.999, but its testing performance slightly decreases in Precision (from 0.944 to 0.931) and F1 score (from 0.944 to 0.939). Despite this, it maintains strong generalization with the highest AUC of 0.981 in testing. RF shows substantial improvement post-tuning, with testing Precision increasing to 0.935 and Recall remaining consistent at 0.938, leading to an F1 score of 0.935 and a well-improved AUC of 0.971. AdaBoost, while improving slightly post-tuning, still underperforms in comparison to the other models, with testing Precision decreasing from 0.935 to 0.931, and a stable F1 score of 0.925, indicating it struggles to match the high performance of other models in high CS.

Across all three classes, CatBoost consistently emerges as the top performer after tuning, particularly excelling in normal and high CS, where it achieves strong generalization and balanced performance across all metrics. LightGBM closely follows, demonstrating excellent generalization in low and high CS but showing slightly reduced effectiveness in normal CS. XGBoost remains a competitive option, particularly in low CS, but shows some signs of overfitting. RF improves significantly after tuning across all classes, especially in terms of Precision and Recall, but still lags behind CatBoost and LightGBM. AdaBoost, despite improvements, consistently underperforms relative to the other models, making it a less reliable choice for this classification task.

Metrics heatmaps

Figure 10 illustrates the heatmaps for each class during the TR and TS stages for the classification metrics (AUC, F1 Score, Recall, Precision, and Accuracy) across the adopted models. For LSC, the heatmaps indicate strong performance across most models post-tuning, with CatBoost, LightGBM, and XGBoost generally leading. Both LightGBM and CatBoost exhibit almost perfect values across all metrics, with near-maximum F1 Scores, Precision, Recall, and AUC values close to 1.0 in the TR stage. During the TS stage, CatBoost continues to outperform, with an AUC of 0.956 and strong values across all metrics, indicating excellent generalization. XGBoost also shows strong performance but slightly trails the CatBoost and LightGBM models in generalization, especially in AUC. RF improves significantly but is still behind the top models, while AdaBoost remains the weakest across all metrics, particularly in F1 Score and Recall.

In NSC, CatBoost and LightGBM continue to dominate the performance charts. CatBoost shows excellent TR and TS performance, particularly with high AUC scores, reaching 0.904 in the TS stage. LightGBM also holds up well, achieving near-perfect metrics during the TR stage and a competitive AUC in the TS stage. XGBoost maintains strong performance but slightly underperforms in recall and precision compared to the top models in the testing set, indicating slight overfitting. RF improves substantially post-tuning, with increased F1 Score and Recall values, yet it still underperforms compared to the CatBoost and LightGBM models. Once again, the AdaBoost model lags behind, particularly in testing metrics, where its recall and precision drop more significantly.

HSC results show a similar trend, with CatBoost leading across most metrics. In the TR stage, the CatBoost, LightGBM, and XGBoost models perform almost perfectly, with AUC values nearing 1.0 and exceptional F1 Scores, Precision, and Recall. In the TS stage, the CatBoost and LightGBM models maintain strong performance, with CatBoost achieving an AUC of 0.964. The XGBoost model also performs well but shows a slight dip in TS stage recall and precision. The RF model continues its improvements, showing stronger performance than AdaBoost in all metrics but still behind the top models. The AdaBoost model remains the weakest model for high CS, especially in the TS stage where it exhibits lower recall and F1 Scores, reflecting weaker generalization.

Across all three classes, The CatBoost and LightGBM consistently outperform the other models in terms of classification performance. These models achieve nearly perfect values across the TR metrics and demonstrate strong generalization capabilities in the TS stage, particularly for low and high CS. The XGBoost model performs well but shows slight signs of overfitting, especially in normal and high CS, where testing results do not perfectly align with training. The RF model significantly improves after tuning but remains less effective than the top models. The AdaBoost model, while improved, remains consistently behind the other models in all classes, making it a less reliable choice for these classification tasks.

Strengths and weaknesses of models

Confusion matrices

Figure 11a shows the confusion matrix for the RF model. The matrix indicates that 148 “Normal” instances were correctly identified, but notable misclassifications occurred, with 13 “Low” instances predicted as “Normal” and 11 “Normal” instances as “Low.” The misclassification between these two adjacent categories suggests difficulty in distinguishing boundary cases, likely due to overlapping feature distributions. The model also struggles with distinguishing “High” from “Normal,” showing a pattern of classification errors in high strength categories. Figure 11b presents the confusion matrix for the AdaBoost model. While this model correctly classified 148 “Normal” instances, misclassification rates in the “Low” and “High” categories increased compared to RF. Seventeen “Low” instances were misclassified as “Normal,” and 12 “Normal” instances were predicted as “Low.” Additionally, 14 “Normal” instances were classified as “High,” indicating a challenge in differentiating borderline cases, likely due to its sensitivity to weak learners, which amplifies errors in certain categories. Figure 11c shows the confusion matrix for the XGBoost model. XGBoost improved overall performance, correctly identifying 154 “Normal” instances and achieving a better balance across all categories compared to RF and AdaBoost. Misclassifications included 13 “Low” instances as “Normal” and 5 “Normal” instances as “Low.” The model correctly classified 22 “High” instances. The ability of XGBoost to reduce misclassification rates suggests that its boosted

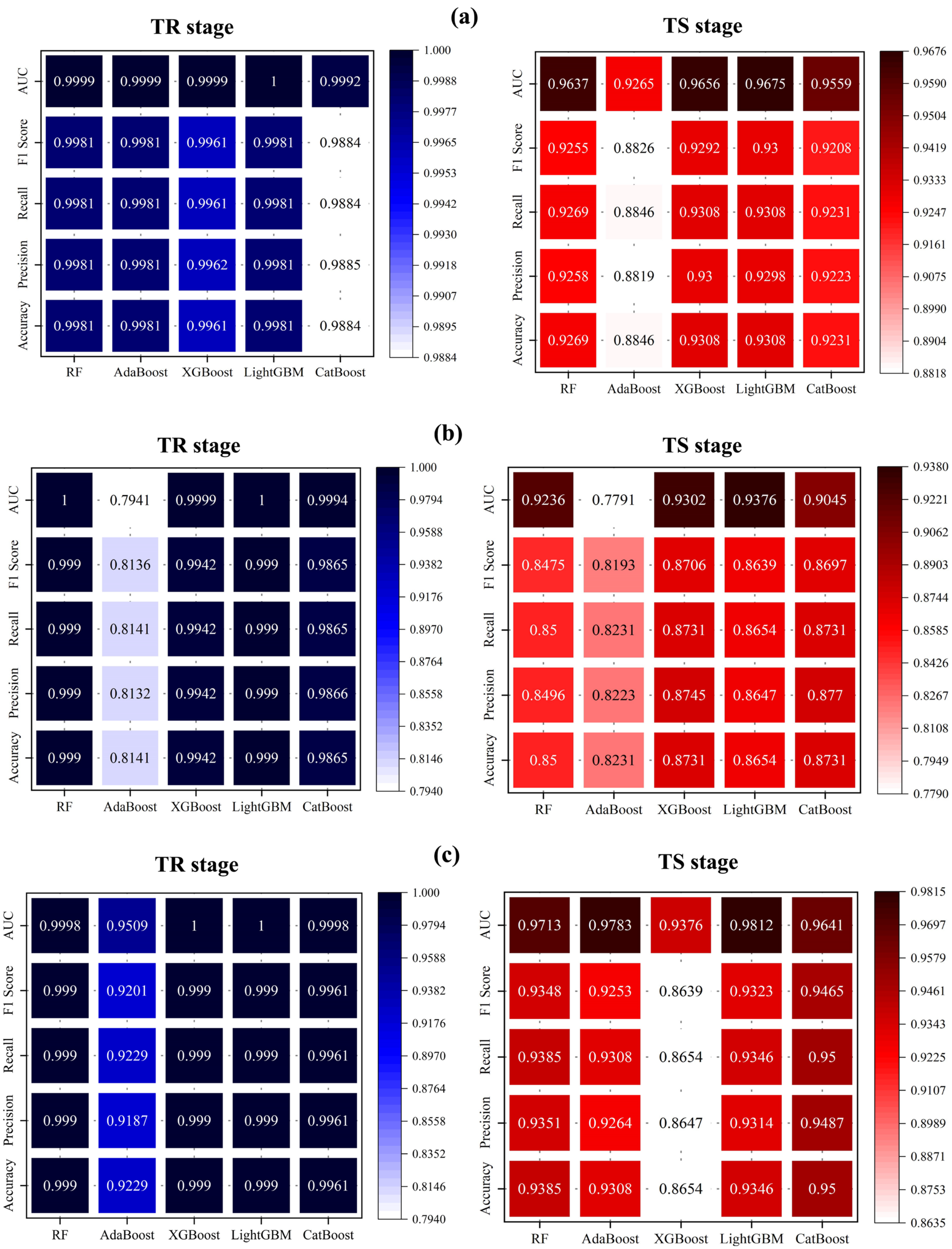

Fig. 10 Heatmap classification metrics during TR and TS stages for (**a**) LSC, **b** NSC, and **c** HSC

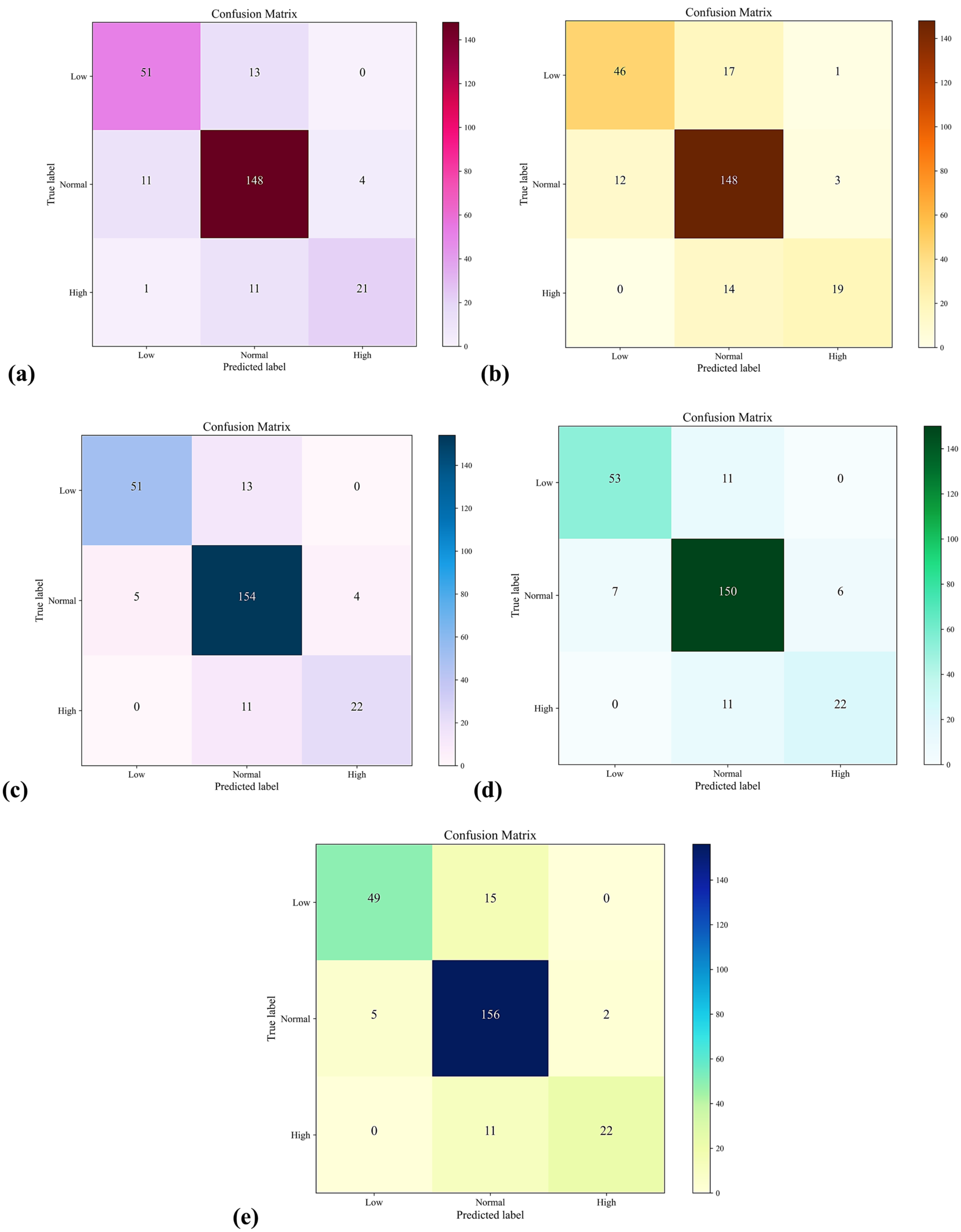

Fig. 11 Confusion matrices showing the classification performance of (**a**) RF, **b** AdaBoost, **c** XGBoost, **d** LightGBM, and **e** CatBoost models in the testing stage across CS classes

decision trees effectively capture complex relationships in the dataset, improving classification performance.

Figure 11d presents the confusion matrix for the LightGBM model, which demonstrated the best overall classification performance. LightGBM correctly identified 150 "Normal" instances and maintained a balanced performance across all categories. Misclassifications included 11 "Low" instances as "Normal" and 7 "Normal" instances as "Low." The model correctly classified 22 "High" instances, showing consistency with XGBoost. The reduced misclassification of LightGBM in "Low" and "High" categories indicates its superior ability to handle imbalanced class distributions and boundary cases, likely due to its gradient-based sampling and exclusive feature bundling techniques. Figure 11e shows the confusion matrix for the CatBoost model. CatBoost correctly classified 156 "Normal" instances, maintaining consistent correct classifications of 22 "High" instances, similar to XGBoost and LightGBM. However, 15 "Low" instances were misclassified as "Normal," and 5 "Normal" instances were misclassified as "Low." While CatBoost had a high classification rate for "Normal" instances, its slightly higher misclassification of "Low" instances compared to LightGBM indicates marginally weaker handling of minority classes. Overall, the LightGBM model stands out as the best-performing model in terms of classification accuracy, misclassification rates, and generalization performance. While CatBoost achieved the highest correct classification for "Normal" instances, LightGBM maintained a better balance across all three strength categories, with fewer errors in misclassifying "Low" and "High" classes.

ROC curves

Figure 12a shows the ROC curves for the RF model. Each curve represents the performance of the model for a specific class (Low, Normal, High). The x-axis shows the False Positive Rate (FPR), while the y-axis shows the True Positive Rate (TPR). The AUC is a measure of the model's ability to distinguish between classes. For the RF model, the AUC values are 0.96 for the "Low" class, 0.95 for the "Normal" class, and 0.97 for the "High" class. This indicates that the model has high discriminatory power, particularly for the "High" class. Figure 12b shows the ROC curves for the AdaBoost model. The AUC values for this model are 0.93 for the "Low" class, 0.78 for the "Normal" class, and 0.98 for the "High" class. While the model performs well for the "Low" and "High" classes, the performance for the "Normal" class is significantly lower, as indicated by the lower AUC value. Figure 12c shows the ROC curves for the XGBoost model. The AUC values for XGBoost are 0.96 for the "Low" class, 0.92 for the "Normal" class, and 0.97 for the "High" class. This model demonstrates strong performance across all classes, with particularly high accuracy for the "Low" and "High" classes. Figure 12d shows the ROC curves for the LightGBM model. The AUC values for LightGBM are 0.97 for the "Low" class, 0.94 for the "Normal" class, and 0.98 for the "High" class. This indicates that LightGBM has excellent performance across all classes, with the highest AUC for the "High" class. Figure 12e shows the ROC curves for the CatBoost model. The AUC values for CatBoost are 0.96 for the "Low" class, 0.90 for the "Normal" class, and 0.96 for the "High" class. While the performance is high across all classes, the AUC for the "Normal" class is slightly lower than the other two classes. In summary, LightGBM appears to be the overall best classifier overall, as it consistently demonstrates the highest AUC values. The model performs exceptionally well for the "High" class with an AUC of 0.98 and maintains strong performance for the "Low" and "Normal" classes, indicating its superior discriminatory power and reliability.

SHAP analysis

Figure 13 presents the SHAP summary dot plots, illustrating the key factors influencing the prediction of concrete CS) across different strength categories. The SHAP analysis for Low-Strength Concrete (LSC), Normal-Strength Concrete (NSC), and High-Strength Concrete (HSC) highlights curing duration (X8) and cement content (X1) as the most significant contributors, consistently exhibiting a strong positive impact on CS. Extended curing enhances strength development, while increased cement content significantly improves overall performance across all strength classifications. For LSC (Fig. 13a), X8 and X1 hold the greatest influence, while superplasticizer (X5) and water content (X4) also contribute, although their effects depend on their proportions. In contrast, blast furnace slag (X2) and fine aggregate (X7) show minimal impact, indicating their limited role in predicting low-strength concrete properties.

In NSC (Fig. 13b), curing duration and cement content remain dominant, but water content (X4) has a significant negative effect when excessive, reinforcing the importance of maintaining an optimal water-to-binder ratio. Coarse aggregate (X6) and superplasticizer moderately affect strength, whereas fly ash (X3) and fine aggregate have negligible influence. For HSC (Fig. 13c), curing duration and cement content continue to be the primary determinants of strength, but superplasticizer (X5) gains prominence, enhancing workability and improving HSC performance. Additionally, coarse aggregate plays a crucial role in structural integrity, while water content requires careful optimization due to its dual effect on strength.

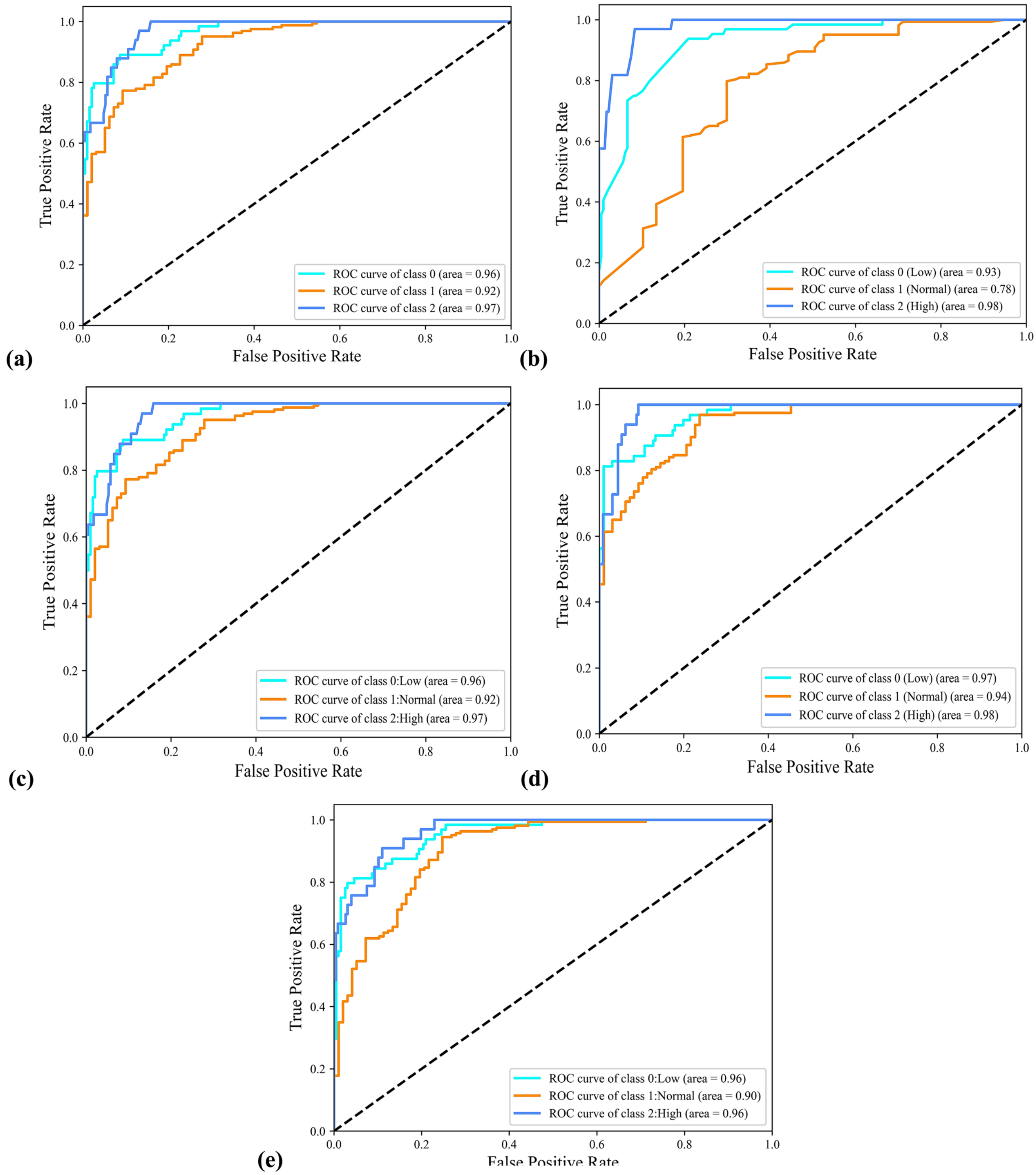

Fig. 12 ROCs showing the classification performance of (**a**) RF, **b** Adaboost, **c** XGBoost, **d** LightGBM, and **e** CatBoost model in the testing stage across the CS classes

Interactive graphical user interface (GUI)

To meet the practical needs of designers seeking efficient application of ML models, this section presents a notable advancement. The complexity involved in database creation, model training, and testing has often posed challenges to the smooth integration of ML into routine design tasks. To overcome these obstacles, a novel solution has been

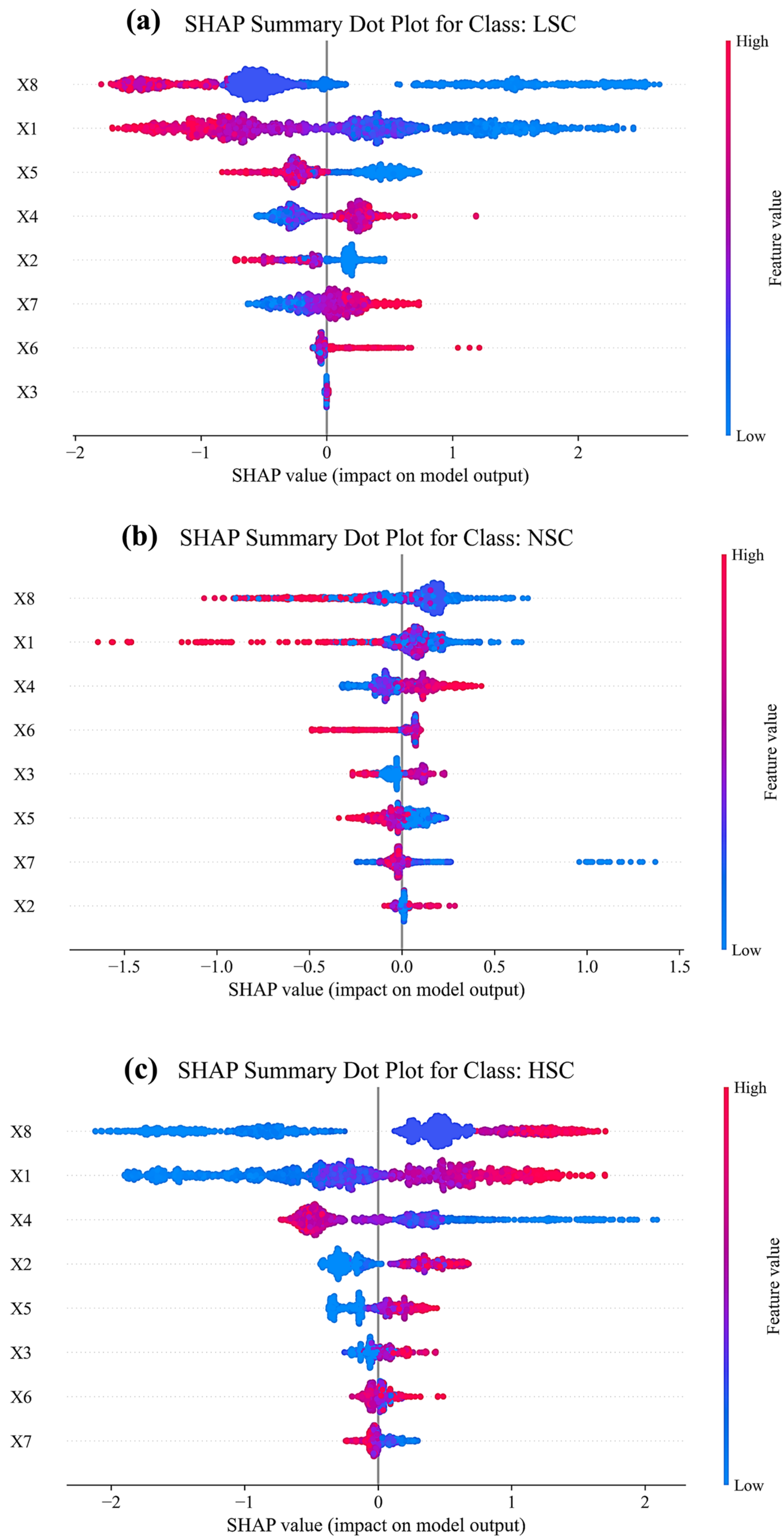

Fig. 13 SHAP charts for (**a**) LSC class; **b** NSC class; **c** HSC class

developed: a Python web application that incorporates a model with optimized hyperparameters through an intuitive GUI, created using the Tkinter package [94, 95]. This interface is tailored for predicting CCS and streamlining the design process, as shown in Fig. 14. By clicking the "**Calculate**" button, users can obtain the predicted CS value. Additionally, the application allows users to select a predictive model from a list to classify concrete based on its CS value. The tool is freely accessible at https://github.com/mkamel24/CS_Class_GUI.

Comparison with previous studies

Comparing the performance of machine learning models with previous studies is crucial for evaluating advancements in predictive accuracy, generalization ability, and computational efficiency. It provides a benchmark for assessing whether newer techniques offer significant improvements over traditional methods and helps validate the effectiveness of different approaches in real-world applications. Natarajan et al. [96] conducted a study on concrete strength classification using various machine learning techniques, including Support Vector Machines (SVM), Bagged Trees, Boosted Trees, and RUSBoosted Trees. Their approach categorized concrete into three strength classes: low, medium, and high, employing ten-fold cross-validation for model evaluation. The study compared these models based on accuracy and area under the curve (AUC), identifying Bagged Trees as the best-performing model, which achieved an overall accuracy of 88.4% across all classes.

In comparison, the present study demonstrates that LightGBM significantly outperforms the models used by Natarajan et al. [96] in both classification accuracy and AUC. After hyperparameter tuning, LightGBM achieved an overall accuracy of 91.0% across all classes, with test accuracies of 93.1% (Class 0), 86.5% (Class 1), and 93.5% (Class 2). Additionally, LightGBM maintained AUC values exceeding 0.96, ensuring robust classification performance. The improvement over Bagged Trees' 88.4% accuracy highlights the effectiveness of the gradient

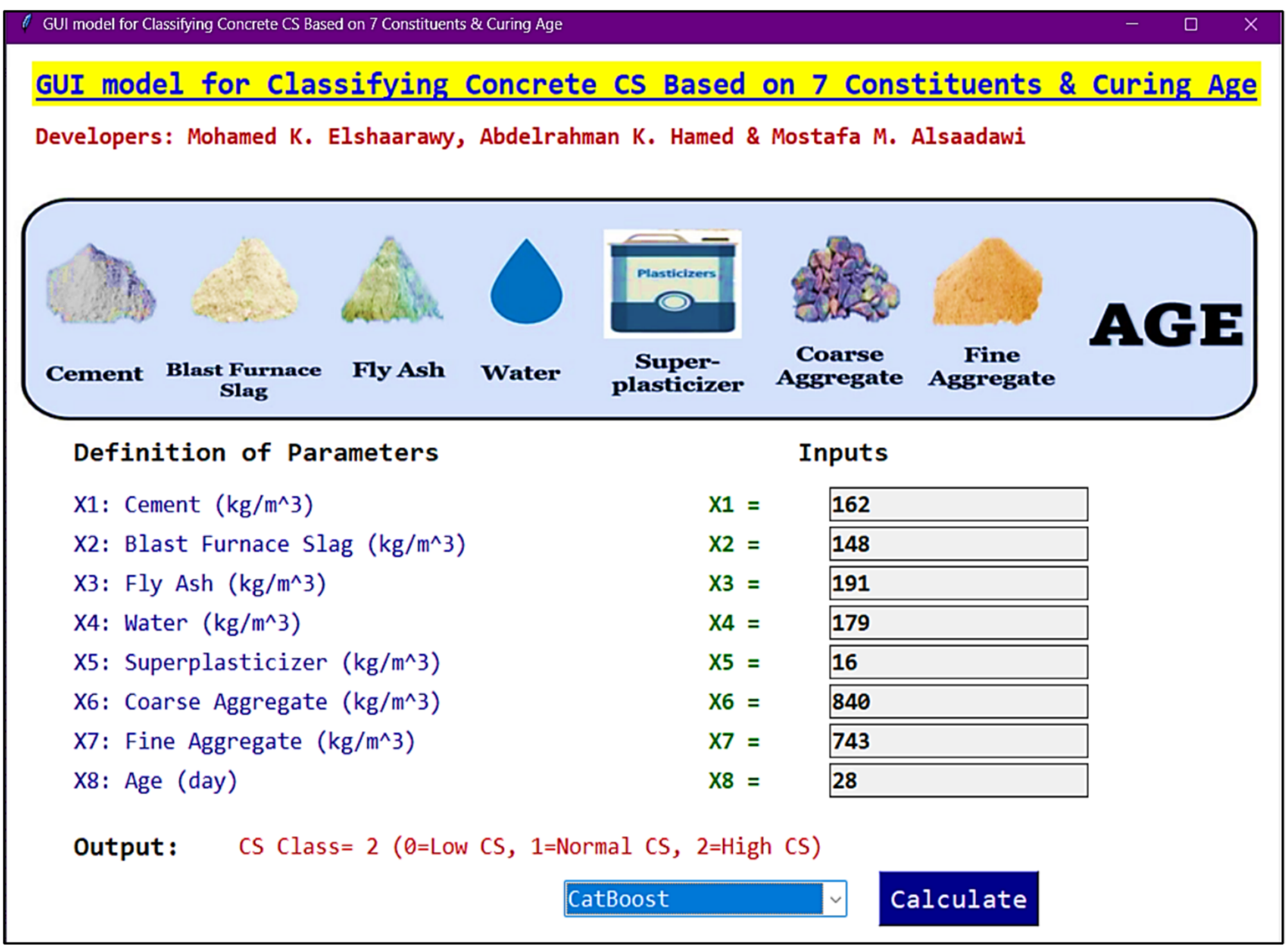

Fig. 14 Developed GUI view for classifying CS of concrete

boosting approach in achieving higher predictive reliability. Overall, the comparison highlights that LightGBM, with advanced gradient boosting techniques and optimized hyperparameters, provides superior classification performance over traditional ensemble bagging methods. This advancement demonstrates the effectiveness of boosting-based approaches for highly accurate and reliable concrete strength classification, offering improved predictive capabilities for real-world applications in concrete quality assessment.

Conclusions

In this study, various machine learning models, including RF, AdaBoost, XGBoost, LightGBM, and CatBoost, were employed to classify concrete compressive strength into distinct categories (Low, Normal, and High). The models were optimized using BO + 5CV to ensure the most effective hyperparameter configurations and enhance generalization performance. The results of the study clearly demonstrate that hyperparameter tuning significantly improves the performance of all models, with particular emphasis on boosting algorithms such as XGBoost, LightGBM, and CatBoost. The following points can be concluded:

- CatBoost and LightGBM emerged as the top-performing models, consistently achieving high accuracy, precision, recall, and F1 scores across all strength classes. Both models demonstrated a superior balance between sensitivity and specificity, as indicated by their strong AUC values. For instance, CatBoost achieved accuracy scores of up to 0.923 for the low strength class and 0.950 for the high strength class after tuning, with corresponding AUC values of 0.956 and 0.964. This model particularly excelled in the Normal strength class, a challenging category for many classifiers, achieving an accuracy of 0.873. The stability and robustness of CatBoost across all categories, evidenced by its low misclassification rates, highlight its suitability for concrete classification tasks.
- XGBoost also demonstrated strong potential, particularly in the low and high strength classes. While RF and AdaBoost exhibited acceptable performance, they showed limitations in handling more complex classification scenarios, particularly in the Normal strength category.
- The use of confusion matrices and ROC curves provided additional insights into the strengths and weaknesses of each model. CatBoost achieved the highest number of correct classifications across all categories, maintaining the most balanced performance with the lowest rates of false positives and false negatives. For instance, CatBoost correctly classified 156 Normal strength instances, more than any other model, and showed exceptional balance in misclassification rates for Low and High strength instances. LightGBM and XGBoost followed closely, demonstrating excellent discriminatory power, particularly for the low and high strength classes, with ROC curves showing AUC values above 0.97 in these categories.
- SHAP analysis confirmed that curing duration and cement content were the most influential factors in predicting compressive strength, with a consistently strong positive impact across all strength classes. Water content and superplasticizer played secondary roles, with their influence varying based on the strength category. In Low-Strength Concrete (LSC), curing duration and cement content were dominant, while superplasticizer and water content also contributed with varying effects. For Normal-Strength Concrete (NSC), excess water content had a significant negative impact, while coarse aggregate and superplasticizer moderately influenced strength. In High-Strength Concrete (HSC), superplasticizer and coarse aggregate gained prominence, reinforcing their role in improving high-strength concrete performance.
- An additional advancement in this study was the development of a user-friendly GUI, specifically designed to bridge the gap between the intricate requirements of machine learning and the practical demands of designers. The interface integrates the CatBoost model, allowing users to input data and obtain real-time strength classifications efficiently.

Limitations and future work

While this study demonstrates the effectiveness of advanced machine learning models like CatBoost, LightGBM, and XGBoost for concrete CS classification, there are several limitations that need to be addressed:

- The dataset used in this study primarily focused on a specific range of concrete types. Expanding it to encompass a wider variety of concrete formulations, such as Lightweight or Ultra-High-Performance concrete, would improve the generalizability of the models.
- Future studies should also investigate the deployment of these models in real-world construction environments, focusing on integration with sensor data and on-site quality control systems to enable continuous monitoring and classification of concrete strength during production.

Acknowledgements The authors would like to thank the journal editor and the anonymous reviewers for their editing and comments.

Authors contribution Conceptualization, M.K.E., A.K.H. and M.M.A.; Methodology, M.K.E.; Software, M.K.E. and A.K.H.; Validation, M.K.E., A.K.H. and M.M.A.; Formal analysis, M.K.E. and M.M.A.; Investigation, A.K.H. and M.M.A.; Resources, M.M.A.; Data curation, M.K.E., A.K.H. and M.M.A.; Writing—original draft preparation, M.K.E. and M.M.A.; Writing—review and editing, M.K.E., A.K.H., and M.M.A.; Visualization, M.K.E. and A.K.H.; Supervision, M.K.E. and M.M.A.; Project administration, M.M.A. All authors have read and agreed to the published version of the manuscript.

Funding Open access funding provided by The Science, Technology & Innovation Funding Authority (STDF) in cooperation with The Egyptian Knowledge Bank (EKB). This research did not receive funding from specific agencies in the public, commercial, or not-for-profit divisions.

Data availability All source data and code supporting the findings of this study are publicly available at https://github.com/mkamel24/CS_Class_GUI.

Declarations

Competing interests The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Human or animal rights This research did not involve human participants or animals.

Informed consent Not applicable.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

1. Hassan HS, Abdel-Gawwad HA, Vásquez-García SR, Israde-Alcántara I, Flores-Ramirez N, Rico JL, Mohammed MS (2019) Cleaner production of one-part white geopolymer cement using pre-treated wood biomass ash and diatomite. J Clean Prod 209:1420–1428. https://doi.org/10.1016/j.jclepro.2018.11.137
2. Liu G, Yang H, Fu Y, Mao C, Xu P, Hong J, Li R (2020) Cyber-physical system-based real-time monitoring and visualization of greenhouse gas emissions of prefabricated construction. J Clean Prod 246:119059. https://doi.org/10.1016/j.jclepro.2019.119059
3. Akbarzadeh Bengar H, Shahmansouri AA (2020) A new anchorage system for CFRP strips in externally strengthened RC continuous beams. J Build Eng 30:101230. https://doi.org/10.1016/j.jobe.2020.101230
4. Benhelal E, Shamsaei E, Rashid MI (2019) Novel modifications in a conventional clinker making process for sustainable cement production. J Clean Prod 221:389–397. https://doi.org/10.1016/j.jclepro.2019.02.259
5. Samimi K, Kamali-Bernard S, Maghsoudi AA, Maghsoudi M, Siad H (2017) Influence of pumice and zeolite on compressive strength, transport properties and resistance to chloride penetration of high strength self-compacting concretes. Construct Build Mater 151:292–311. https://doi.org/10.1016/j.conbuildmat.2017.06.071
6. Taji I, Ghorbani S, de Brito J, Tam VWY, Sharifi S, Davoodi A, Tavakkolizadeh M (2019) Application of statistical analysis to evaluate the corrosion resistance of steel rebars embedded in concrete with marble and granite waste dust. J Clean Prod 210:837–846. https://doi.org/10.1016/j.jclepro.2018.11.091
7. Barcelo L, Kline J, Walenta G, Gartner E (2014) Cement and carbon emissions. Mater Struct 47:1055–1065. https://doi.org/10.1617/s11527-013-0114-5
8. Kajaste R, Hurme M (2016) Cement industry greenhouse gas emissions–management options and abatement cost. J Clean Prod 112:4041–4052. https://doi.org/10.1016/j.jclepro.2015.07.055
9. Czarnecki S, Shariq M, Nikoo M, Sadowski Ł (2021) An intelligent model for the prediction of the compressive strength of cementitious composites with ground granulated blast furnace slag based on ultrasonic pulse velocity measurements. Measurement 172:108951. https://doi.org/10.1016/j.measurement.2020.108951
10. Chajec A (2021) Granite powder versus fly ash for the sustainable production of air-cured cementitious mortars. Materials 14:1208. https://doi.org/10.3390/ma14051208
11. Shubbar AA, Jafer H, Dulaimi A, Hashim K, Atherton W, Sadique M (2018) The development of a low carbon binder produced from the ternary blending of cement, ground granulated blast furnace slag and high calcium fly ash: an experimental and statistical approach. Constr Build Mater 187:1051–1060. https://doi.org/10.1016/j.conbuildmat.2018.08.021
12. Phul AA, Memon MJ, Shah SNR, Sandhu AR (2019) GGBS and fly ash effects on compressive strength by partial replacement of cement concrete. Civil Eng J 5(4):913–921. https://doi.org/10.28991/cej-2019-03091299
13. Torres A, Bartlett L, Pilgrim C (2017) Effect of foundry waste on the mechanical properties of Portland cement concrete. Constr Build Mater 135:674–681. https://doi.org/10.1016/j.conbuildmat.2017.01.028
14. Elrefaei AEMM, Alsaadawi MM, Wagdy W (2023) Characteristics of high-strength concrete reinforced with steel fibers recovered from waste tires. Key Eng Mater 945:145–156. https://doi.org/10.4028/p-d5v1nm
15. Wangler T, Roussel N, Bos FP, Salet TAM, Flatt RJ (2019) Digital concrete: a review. Cem Concr Res 123:105780. https://doi.org/10.1016/j.cemconres.2019.105780
16. Alsaadawi MM, Amin M, Tahwia AM (2022) Thermal, mechanical and microstructural properties of sustainable concrete incorporating phase change materials. Constr Build Mater 356:129300. https://doi.org/10.1016/j.conbuildmat.2022.129300
17. Elrefaei AE, Alsaadawi M, Elshafiey MM, Abdolwahab M, Oan AF, (2024) Performance evaluation of ultra high performance concrete manufactured with recycled steel fiber, in: 2024. pp. 3–13. https://doi.org/10.4028/p-dWhX1H.
18. Khaloo AR, Dehestani M, Rahmatabadi P (2008) Mechanical properties of concrete containing a high volume of tire–rubber particles. Waste Manage 28:2472–2482. https://doi.org/10.1016/j.wasman.2008.01.015
19. Song H, Ahmad A, Farooq F, Ostrowski KA, Maślak M, Czarnecki S, Aslam F (2021) Predicting the compressive strength of concrete with fly ash admixture using machine learning

algorithms. Constr Build Mater 308:125021. https://doi.org/10.1016/j.conbuildmat.2021.125021
20. Cotsovos DM, Pavlović MN (2008) Numerical investigation of concrete subjected to compressive impact loading. Part 2: parametric investigation of factors affecting behaviour at high loading rates. Comput Struct 86:164–180. https://doi.org/10.1016/j.compstruc.2007.05.015
21. Li M, Hao H, Shi Y, Hao Y (2018) Specimen shape and size effects on the concrete compressive strength under static and dynamic tests. Constr Build Mater 161:84–93. https://doi.org/10.1016/j.conbuildmat.2017.11.069
22. Bhanja S, Sengupta B (2002) Investigations on the compressive strength of silica fume concrete using statistical methods. Cem Concr Res 32:1391–1394. https://doi.org/10.1016/S0008-8846(02)00787-1
23. Bharatkumar B, Narayanan R, Raghuprasad B, Ramachandramurthy D (2001) Mix proportioning of high performance concrete. Cement Concr Compos 23:71–80. https://doi.org/10.1016/S0958-9465(00)00071-8
24. Zain MFM, Abd SM (2008) Multiple regression model for compressive strength prediction of high performance concrete. J Appl Sci 9:155–160. https://doi.org/10.3923/jas.2009.155.160
25. Li D, Tang Z, Kang Q, Zhang X, Li Y (2023) Machine learning-based method for predicting compressive strength of concrete. Processes 11:390. https://doi.org/10.3390/pr11020390
26. Salehi H, Burgueño R (2018) Emerging artificial intelligence methods in structural engineering. Eng Struct 171:170–189. https://doi.org/10.1016/j.engstruct.2018.05.084
27. Zhou G, Moayedi H, Bahiraei M, Lyu Z (2020) Employing artificial bee colony and particle swarm techniques for optimizing a neural network in prediction of heating and cooling loads of residential buildings. J Clean Prod 254:120082. https://doi.org/10.1016/j.jclepro.2020.120082
28. Das P, Kashem A, Hasan I, Islam M (2024) A comparative study of machine learning models for construction costs prediction with natural gradient boosting algorithm and SHAP analysis. Asian J Civil Eng 25:3301–3316. https://doi.org/10.1007/s42107-023-00980-z
29. Shaban WM, Elbaz K, Yang J, Thomas BS, Shen X, Li L, Du Y, Xie J, Li L (2021) Effect of pozzolan slurries on recycled aggregate concrete: mechanical and durability performance. Constr Build Mater 276:121940. https://doi.org/10.1016/j.conbuildmat.2020.121940
30. Shaban WM, Daef KS (2023) Performance of eco-friendly concrete: a safe direction to sustainable cities development. Smart Construct Sustain Cities 1:13. https://doi.org/10.1007/s44268-023-00015-1
31. Xu C, Zhang Y, Isleem HF, Qiu D, Zhang Y, Alsaadawi MM, Tipu RK, El-Demerdash WE, Hamed AY (2024) Numerical and machine learning models for concentrically and eccentrically loaded CFST columns confined with FRP wraps. Struct Concr. https://doi.org/10.1002/suco.202400541
32. Elshaarawy MK, Alsaadawi MM, Hamed AK (2024) Machine learning and interactive GUI for concrete compressive strength prediction. Sci Rep 14:16694. https://doi.org/10.1038/s41598-024-66957-3
33. Chithra S, Kumar SRRS, Chinnaraju K, Alfin Ashmita F (2016) A comparative study on the compressive strength prediction models for high performance concrete containing nano silica and copper slag using regression analysis and artificial neural networks. Construct Build Mater 114:528–535. https://doi.org/10.1016/j.conbuildmat.2016.03.214
34. Nguyen H, Vu T, Vo TP, Thai H-T (2021) Efficient machine learning models for prediction of concrete strengths. Constr Build Mater 266:120950. https://doi.org/10.1016/j.conbuildmat.2020.120950
35. Kumar A, Arora HC, Kapoor NR, Mohammed MA, Kumar K, Majumdar A, Thinnukool O (2022) Compressive strength prediction of lightweight concrete: machine learning models. Sustainability 14:2404. https://doi.org/10.3390/su14042404
36. Ashrafian A, Taheri Amiri MJ, Rezaie-Balf M, Ozbakkaloglu T, Lotfi-Omran O (2018) Prediction of compressive strength and ultrasonic pulse velocity of fiber reinforced concrete incorporating nano silica using heuristic regression methods. Construct Build Mater 190:479–494. https://doi.org/10.1016/j.conbuildmat.2018.09.047
37. Zhang J, Ma G, Huang Y, Sun J, Aslani F, Nener B (2019) Modelling uniaxial compressive strength of lightweight self-compacting concrete using random forest regression. Constr Build Mater 210:713–719. https://doi.org/10.1016/j.conbuildmat.2019.03.189
38. Aslam F, Farooq F, Amin MN, Khan K, Waheed A, Akbar A, Javed MF, Alyousef R, Alabdulijabbar H (2020) Applications of gene expression programming for estimating compressive strength of high-strength concrete. Adv Civil Eng 2020:1–23. https://doi.org/10.1155/2020/8850535
39. Golafshani EM, Behnood A, Arashpour M (2020) Predicting the compressive strength of normal and high-performance concretes using ANN and ANFIS hybridized with grey wolf optimizer. Constr Build Mater 232:117266. https://doi.org/10.1016/j.conbuildmat.2019.117266
40. Song Y, Zhao J, Ostrowski KA, Javed MF, Ahmad A, Khan MI, Aslam F, Kinasz R (2021) Prediction of compressive strength of fly-ash-based concrete using ensemble and non-ensemble supervised machine-learning approaches. Appl Sci 12:361. https://doi.org/10.3390/app12010361
41. Wang M, Kang J, Liu W, Su J, Li M (2022) Research on prediction of compressive strength of fly ash and slag mixed concrete based on machine learning. PLoS ONE 17:e0279293. https://doi.org/10.1371/journal.pone.0279293
42. Ahmad A, Farooq F, Niewiadomski P, Ostrowski K, Akbar A, Aslam F, Alyousef R (2021) Prediction of compressive strength of fly ash based concrete using individual and ensemble algorithm. Materials 14:794. https://doi.org/10.3390/ma14040794
43. Yu Y, Li W, Li J, Nguyen TN (2018) A novel optimised self-learning method for compressive strength prediction of high performance concrete. Constr Build Mater 184:229–247. https://doi.org/10.1016/j.conbuildmat.2018.06.219
44. Islam N, Kashem A, Das P, Ali MN, Paul S (2024) Prediction of high-performance concrete compressive strength using deep learning techniques. Asian J Civil Eng 25:327–341. https://doi.org/10.1007/s42107-023-00778-z
45. Huang J, Sabri MMS, Ulrikh DV, Ahmad M, Alsaffar KAM (2022) Predicting the compressive strength of the cement-fly ash-slag ternary concrete using the firefly algorithm (FA) and random forest (RF) hybrid machine-learning method. Materials 15:4193. https://doi.org/10.3390/ma15124193
46. Bui D-K, Nguyen T, Chou J-S, Nguyen-Xuan H, Ngo TD (2018) A modified firefly algorithm-artificial neural network expert system for predicting compressive and tensile strength of high-performance concrete. Constr Build Mater 180:320–333. https://doi.org/10.1016/j.conbuildmat.2018.05.201
47. Kashem A, Das P (2023) Compressive strength prediction of high-strength concrete using hybrid machine learning approaches by incorporating SHAP analysis. Asian J Civil Eng 24:3243–3263. https://doi.org/10.1007/s42107-023-00707-0
48. Zhang J, Zhao Y, Prediction of compressive strength of ultra-high performance concrete (UHPC) containing supplementary cementitious materials, in: 2017 International Conference on Smart Grid and Electrical Automation (ICSGEA), IEEE, 2017: pp. 522–525. https://doi.org/10.1109/ICSGEA.2017.150.
49. Das P, Kashem A (2024) Hybrid machine learning approach to prediction of the compressive and flexural strengths of UHPC

and parametric analysis with shapley additive explanations. Case Studies Construct Mater 20:e02723. https://doi.org/10.1016/j.cscm.2023.e02723
50. Kashem A, Karim R, Malo SC, Das P, Datta SD, Alharthai M (2024) Hybrid data-driven approaches to predicting the compressive strength of ultra-high-performance concrete using SHAP and PDP analyses. Case Studies Construct Mater 20:e02991. https://doi.org/10.1016/j.cscm.2024.e02991
51. Al-Mughanam T, Aldhyani THH, Alsubari B, Al-Yaari M (2020) Modeling of compressive strength of sustainable self-compacting concrete incorporating treated palm oil fuel ash using artificial neural network. Sustainability 12:9322. https://doi.org/10.3390/su12229322
52. Al-Hashem MN, Amin MN, Raheel M, Khan K, Alkadhim HA, Imran M, Ullah S, Iqbal M (2022) Predicting the compressive strength of concrete containing fly ash and rice husk ash using ANN and GEP models. Materials 15:7713. https://doi.org/10.3390/ma15217713
53. Kashem A, Karim R, Das P, Datta SD, Alharthai M (2024) Compressive strength prediction of sustainable concrete incorporating rice husk ash (RHA) using hybrid machine learning algorithms and parametric analyses. Case Studies Construct Mater 20:e03030. https://doi.org/10.1016/j.cscm.2024.e03030
54. Paul S, Das P, Kashem A, Islam N (2024) Sustainable of rice husk ash concrete compressive strength prediction utilizing artificial intelligence techniques. Asian J Civil Eng 25:1349–1364. https://doi.org/10.1007/s42107-023-00847-3
55. Karim R, Islam MH, Datta SD, Kashem A (2024) Synergistic effects of supplementary cementitious materials and compressive strength prediction of concrete using machine learning algorithms with SHAP and PDP analyses. Case Studies Construct Mater 20:e02828. https://doi.org/10.1016/j.cscm.2023.e02828
56. I.-C. Yeh, (2007) Concrete compressive strength, UCI Machine Learning Repository, https://doi.org/10.24432/C5PK67.
57. Eltarabily MG, Abd-Elhamid HF, Zeleňáková M, Elshaarawy MK, Elkiki M, Selim T (2023) Predicting seepage losses from lined irrigation canals using machine learning models. Frontiers Water. https://doi.org/10.3389/frwa.2023.1287357
58. Ren Y, Isleem HF, Almoghaye WJK, Hamed AK, Jangir P, Arpita, Tejani GG, Ezugwu AE, Soliman AA (2025) Machine learning-based prediction of elliptical double steel columns under compression loading. J Big Data 12:50. https://doi.org/10.1186/s40537-025-01081-1
59. Ergen F, Katlav M (2024) Machine and deep learning-based prediction of flexural moment capacity of ultra-high performance concrete beams with/out steel fiber. Asian J Civil Eng. https://doi.org/10.1007/s42107-024-01064-2
60. Paudel S, Pudasaini A, Shrestha RK, Kharel E (2023) Compressive strength of concrete material using machine learning techniques. Cleaner Eng Technol 15:100661. https://doi.org/10.1016/j.clet.2023.100661
61. Sánchez ER, Santacruz EFV, Maceda HC (2023) Effort and cost estimation using decision tree techniques and story points in agile software development. Mathematics 11(6):1477. https://doi.org/10.3390/math11061477
62. Chen T, Guestrin C, Xgboost: A scalable tree boosting system, in: Proceedings of the 22nd Acm Sigkdd International Conference on Knowledge Discovery and Data Mining, 2016: pp. 785–794.
63. Eltarabily MG, Hamed AK, Elkiki M, Selim T (2024) Hydraulic assessment of different types of piano key weirs. ISH J Hydra Eng. https://doi.org/10.1080/09715010.2024.2415938
64. Wang D, ZhangY , Zhao Y, LightGBM: an effective miRNA classification method in breast cancer patients, in: Proceedings of the 2017 International Conference on Computational Biology and Bioinformatics, 2017: pp. 7–11
65. Yang S, Zhang H (2018) Comparison of several data mining methods in credit card default prediction. Intell Inf Manag 10:115–122. https://doi.org/10.4236/iim.2018.105010
66. AV Dorogush, V Ershov, A Gulin, CatBoost: gradient boosting with categorical features support, ArXiv Preprint. (2018).
67. Hancock JT, Khoshgoftaar TM (2020) CatBoost for big data: an interdisciplinary review. J Big Data 7:94. https://doi.org/10.1186/s40537-020-00369-8
68. Zhang J, Almoghayer WJK, Isleem HF, Negi BS, Mahmoud HA, Elshaarawy MK (2025) Machine learning for the prediction of the axial load-carrying capacity of FRP reinforced hollow concrete column. Struct Concrete. https://doi.org/10.1002/suco.202400886
69. Elshaarawy MK, Armanuos AM (2025) Predicting seawater intrusion wedge length in coastal aquifers using hybrid gradient boosting techniques. Earth Sci Inf. https://doi.org/10.1007/s12145-025-01755-7
70. Elshaarawy MK (2025) Stacked-based hybrid gradient boosting models for estimating seepage from lined canals. J Water Process Eng. https://doi.org/10.1016/j.jwpe.2024.106913
71. Shen F, Jha I, Isleem HF, Almoghayer WJK, Khishe M, Elshaarawy MK (2025) Advanced predictive machine and deep learning models for round-ended CFST column. Sci Rep 15:6194. https://doi.org/10.1038/s41598-025-90648-2
72. Hamed AK, Elshaarawy MK, Alsaadawi MM (2025) Stacked-based machine learning to predict the uniaxial compressive strength of concrete materials. Comput Struct 308:107644. https://doi.org/10.1016/j.compstruc.2025.107644
73. Pandey M, Karbasi M, Jamei M, Malik A, Pu JH (2023) A comprehensive experimental and computational investigation on estimation of scour depth at bridge abutment: emerging ensemble intelligent systems. Water Resour Manage 37:3745–3767. https://doi.org/10.1007/s11269-023-03525-w
74. Luat N-V, Han SW, Lee K (2021) Genetic algorithm hybridized with eXtreme gradient boosting to predict axial compressive capacity of CCFST columns. Compos Struct 278:114733. https://doi.org/10.1016/j.compstruct.2021.114733
75. Snoek J, Larochelle H, Adams RP, (2012) Practical bayesian optimization of machine learning algorithms, Advances in Neural Information Processing Systems. 25
76. Zhang W, Wu C, Zhong H, Li Y, Wang L (2021) Prediction of undrained shear strength using extreme gradient boosting and random forest based on Bayesian optimization. Geosci Front 12:469–477. https://doi.org/10.1016/j.gsf.2020.03.007
77. Albahli S (2023) Efficient hyperparameter tuning for predicting student performance with Bayesian optimization. Multimed Tools Appl 83:52711–52735. https://doi.org/10.1007/s11042-023-17525-w
78. Joy TT, Rana S, Gupta S, Venkatesh S (2020) Fast hyperparameter tuning using Bayesian optimization with directional derivatives. Knowl-Based Syst 205:106247. https://doi.org/10.1016/j.knosys.2020.106247
79. Maslov KA (2024) Bayesian optimization with time-decaying jitter for hyperparameter tuning of neural networks. In: Yavorskiy R, Cavalli AR, Kalenkova A (eds) Tools and methods of program analysis: 6th International Conference, TMPA 2021, Tomsk, Russia, November 25–27, 2021, Revised Selected Papers. Springer Nature Switzerland, Cham, pp 26–40. https://doi.org/10.1007/978-3-031-50423-5_3
80. Victoria AH, Maragatham G (2021) Automatic tuning of hyperparameters using Bayesian optimization. Evol Syst 12:217–223. https://doi.org/10.1007/s12530-020-09345-2
81. Tian W, Isleem HF, Hamed AK, Elshaarawy MK (2024) Enhssancing discharge prediction over Type-A piano key weirs: an innovative machine learning approach. Flow Meas Instrum 100:102732. https://doi.org/10.1016/j.flowmeasinst.2024.102732

82. Elshaarawy MK, Hamed AK (2024) Stacked ensemble model for optimized prediction of triangular side orifice discharge coefficient. Eng Optim. https://doi.org/10.1080/0305215X.2024.2397431
83. Kumar M, Sihag P, Tiwari NK, Ranjan S (2020) Experimental study and modelling discharge coefficient of trapezoidal and rectangular piano key weirs. Appl Water Sci 10:43
84. Najah Ahmed A, Binti Othman F, Abdulmohsin Afan H, Khaleel Ibrahim R, Ming Fai C, Shabbir Hossain M, Ehteram M, Elshafie A (2019) Msachine learning methods for better water quality prediction. J Hydrology. https://doi.org/10.1016/j.jhydrol.2019.124084
85. Widodo S, Brawijaya H, Samudi S (2022) Stratified K-fold cross validation optimization on machine learning for prediction. Sinkron 7(4):2407–2414. https://doi.org/10.33395/sinkron.v7i4.11792
86. Mahesh TR, Vinoth Kumar V, Dhilip Kumar V, Geman O, Margala M, Guduri M (2023) The stratified K-folds cross-validation and class-balancing methods with high-performance ensemble classifiers for breast cancer classification. Healthcare Anal 4:100247. https://doi.org/10.1016/j.health.2023.100247
87. Prusty S, Patnaik S, Dash SK (2022) SKCV: stratified K-fold cross-validation on ML classifiers for predicting cervical cancer. Front Nanotechnol. https://doi.org/10.3389/fnano.2022.972421
88. Elshaarawy MK, Hamed AK (2025) Modeling hydraulic jump roller length on rough beds: a comparative study of ANN and GEP models. J Umm Al-Qura Univ Eng Archit. https://doi.org/10.1007/s43995-024-00093-x
89. Elshaarawy MK, Hamed AK (2024) Machine learning and interactive GUI for estimating roller length of hydraulic jumps. Neural Comput Appl. https://doi.org/10.1007/s00521-024-10846-3
90. Elshaarawy MK, Elmasry NH (2024) Experimental and numerical modeling of seepage in trapezoidal channels. Knowledge-Based Eng Sci 5(3):43–60. https://doi.org/10.51526/kbes.2024.5.3.43-60s
91. Shang L, Isleem HF, Alsaadawi MM (2025) Deep learning-based modelling of polyvinyl chloride tube-confined concrete columns under different load eccentricities. Eng Appl Artif Intell 145:110217. https://doi.org/10.1016/j.engappai.2025.110217
92. Hamed AK, Elshaarawy MK (2025) Soft computing approaches for forecasting discharge over symmetrical piano key weirs. AI in Civil Eng. https://doi.org/10.1007/s43503-024-00048-0
93. Lundh F, An introduction to tkinter, URL: Www. Pythonware. Com/Library/Tkinter/Introduction/Index. Htm. 539 (1999) 540.
94. Pant A, Ramana GV (2022) Prediction of pullout interaction coefficient of geogrids by extreme gradient boosting model. Geotextiles Geomembranes 50:1188–1198. https://doi.org/10.1016/j.geotexmem.2022.08.003
95. Isleem HF, Qiong T, Alsaadawi MM, Elshaarawy MK, Mansour DM, Abdullah F, Mandor A, Sor NH, Jahami A (2024) Numerical and machine learning modeling of GFRP confined concrete-steel hollow elliptical columns. Sci Rep 14:18647. https://doi.org/10.1038/s41598-024-68360-4
96. Natarajan N, Sundar MS, Berlin M, Vasudevan M, (2022) Concrete classification using machine learning techniques BT–Advances in Structural Mechanics and Applications, in: J.A. Fonseca de Oliveira Correia, S. Choudhury, S. Dutta (Eds.), Springer International Publishing, Cham, pp. 179–193